

OPERATING SYSTEMS

DESKTOP



Windows



Mac OS X



Linux

MOBILE



PCCST403-OPERATING SYSTEMS

—

Module 3

TKMIT,KOLLAM

—

SAJANA.A,AP CSE
TKMIT,KOLLAM

Syllabus

I/O system: Modern System architecture, Programmed I/O, Interrupts, DMA, Device interaction methods, The Device Driver (Book 1 Ch 36),

Hard disk: Geometry (Book 1 Ch 37), disk scheduling (Book 3 Section 11.2)

Case study: Linux I/O schedulers - Elevator, Complete Fair Queuing (Book 2 Ch 14)

Files and Directories: The File System Interface - File descriptor, reading and writing files (sequential and random access), Removing files - Hard links and Symbolic links, Creating, reading and deleting directories, Permission bits and Access Control Lists, Mounting a file system (Book 1 Ch 39) .

File Organization: The Inode, The Multi-Level Index (Book 1 Ch 40) Case study: VFS Objects and Their Data Structures - The Inode Object, Inode Operations (Book 2 Ch 13)

Introduction-I/O system

Persistence

- RAM is Volatile - Again we need two parts here
- **Hardware** and **software** are needed to store data persistently
 - Hardware: I/O devices such as hard drive, SSDs, etc.
 - Software:
 - File system manages the disk
 - File system is responsible for any files that the user creates
 - Read, writes are handled by file system which interacts with low level device drivers



I/O Systems: Bridging the Speed Gap

Input makes systems useful. Output makes them usable. The OS must manage both without wasting the CPU.

THE CORE PROBLEM:

Imagine a program with no input—it produces the same result every time. Imagine a program with no output—it has no purpose.

I/O is critical, but it poses a massive architectural challenge.

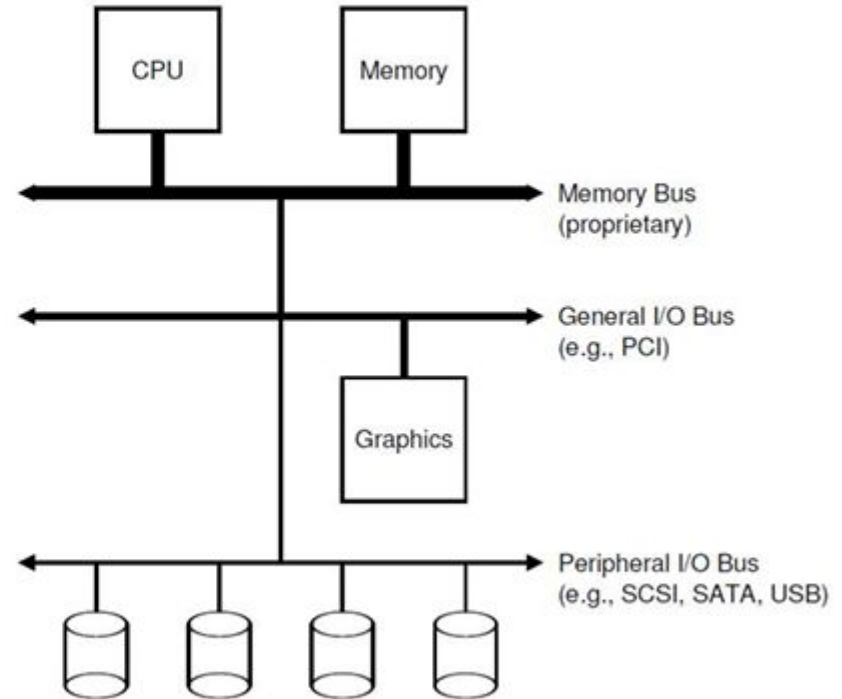


CRUX: How do we integrate slow, physically distant devices into a high-speed system? How do we make them efficient?

Input/Output Devices

- CPU and memory connected via high speed system (memory) bus
 - **Bus** = set of wires carrying signals
- I/O devices connect to the CPU and memory via other separate buses
 - High speed bus, e.g., PCI
 - Other: SCSI, USB, SATA
- Point of connection to the system: **port**

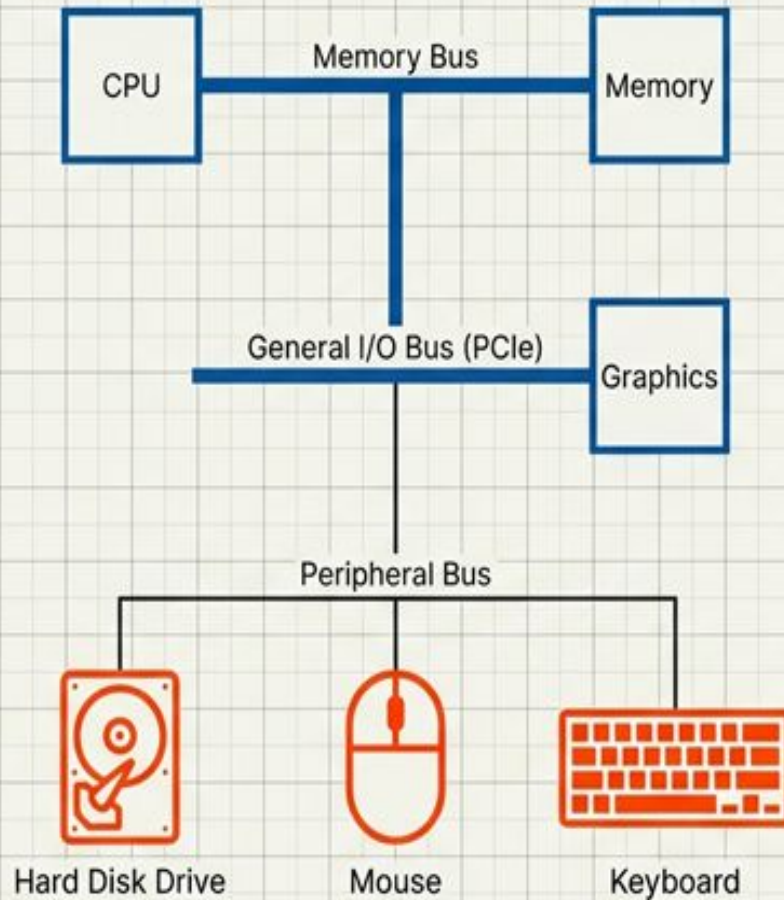
PROTOTYPICAL SYSTEM ARCHITECTURE



High Cost /
Low Latency /
Short Distance

Distance & Cost

Low Cost /
High Latency /
Long Distance



The Suburbs: Slow devices must be relegated to peripheral buses to preserve the signal integrity of the high-speed city center.

1. MODERN SYSTEM ARCHITECTURE

- modern systems increasingly use specialized chipsets and faster point-to-point interconnects to improve performance.
- CPU connects most closely to the memory system, but also has a high-performance connection to the graphics card

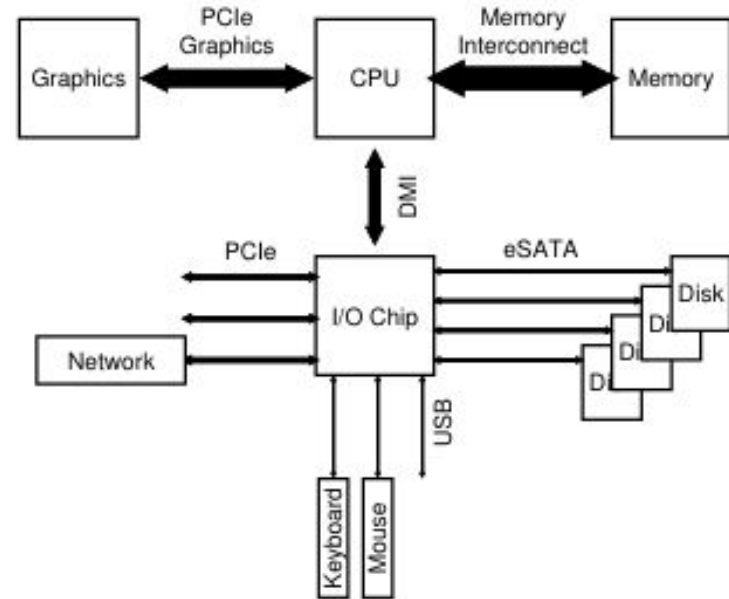
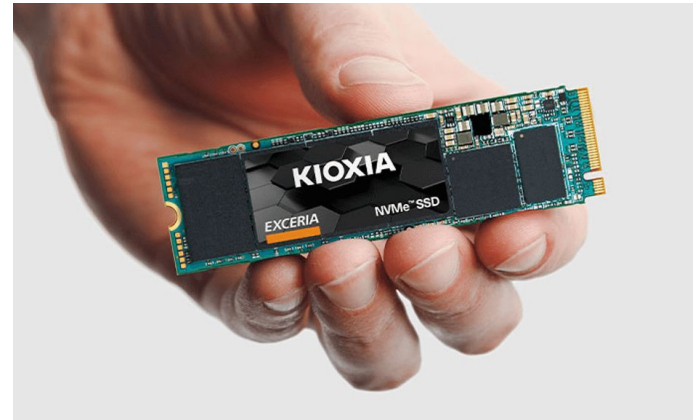
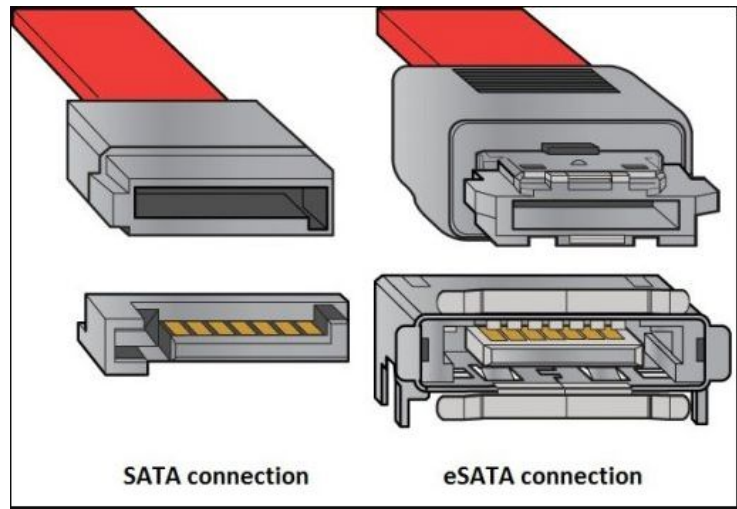
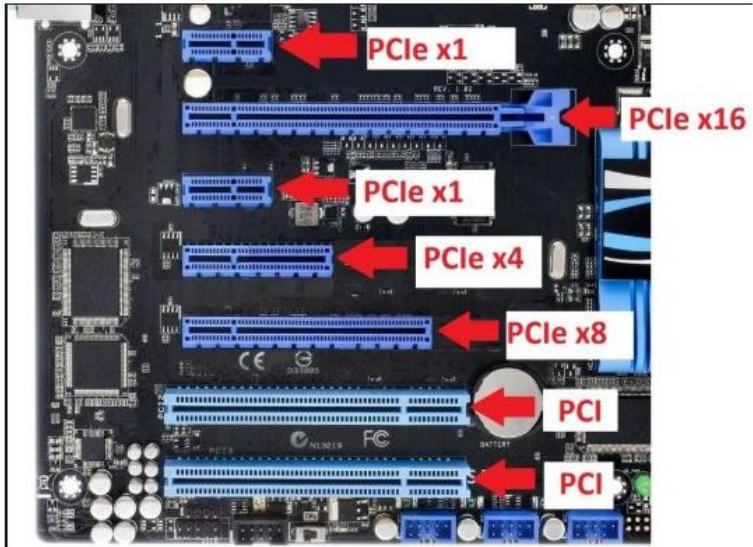


Figure 36.2: Modern System Architecture

- The CPU connects to an I/O chip via Intel's proprietary DMI (Direct Media Interface), and the rest of the devices connect to this chip via a number of different interconnects
- hard drives connect to the system via the eSATA interface(for external SATA)
- Below the I/O chip are a number of USB (Universal Serial Bus) connections, which enable a keyboard and mouse to be attached to the computer. On many modern systems, USB is used for low performance devices.
- higher performance devices can be connected to the system via PCIe(Peripheral Component Interconnect Express). In this diagram, a network interface is attached to the system here; higher performance storage devices (such as NVMe persistent storage devices) are often connected here.



Simple Device Model

- Block devices store a set of numbered blocks (disks)
- Character devices produce/consume stream of bytes (keyboard)
- Devices expose an interface of memory registers
 - Current status of device
 - Command to execute
 - Data to transfer
- Device controller manages device, internals of device are usually hidden

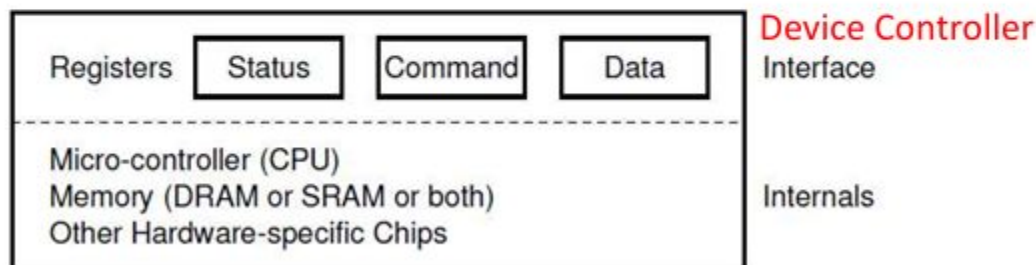
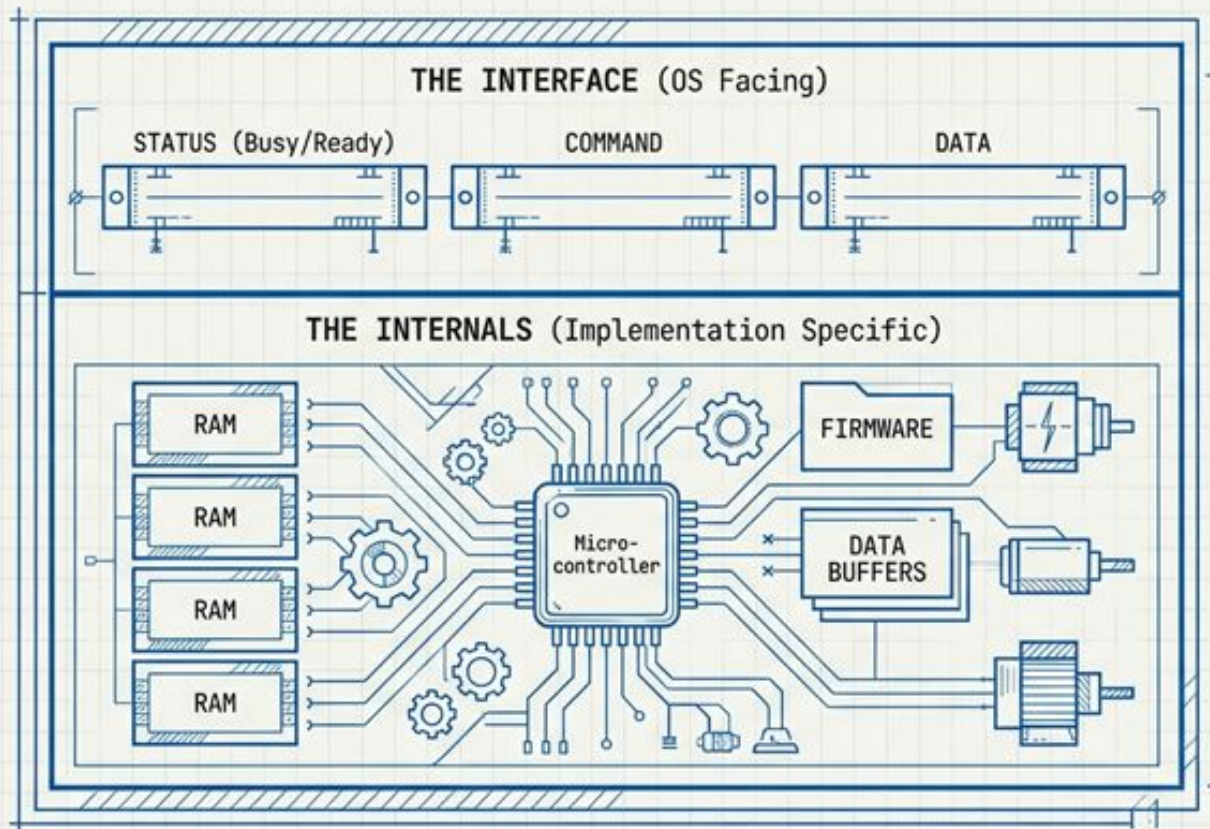


Figure 36.2: A Canonical Device

The Canonical Device



The Exterior: Universal.

To the OS, every device is just a set of registers.

The Interior: Specific.

A RAID controller might have 100k lines of firmware inside, but the interface remains simple.

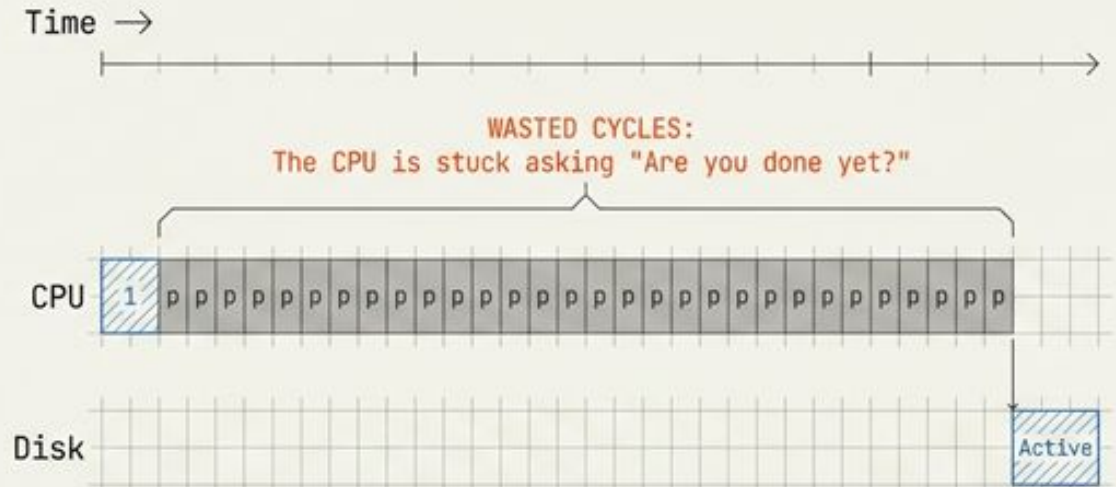
A simple execution of I/O requests

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (Doing so starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

- Simple model of I/O, e.g., read/write block of data from disk
 - Give command to device via command register
 - Write: transfer data to device via data register
 - Poll status register to see if I/O operation completes
 - Read: Copy data from data register to main memory after I/O completes
- Polling status to see if device ready constantly – wastes CPU cycles

The Naïve Protocol: Polling

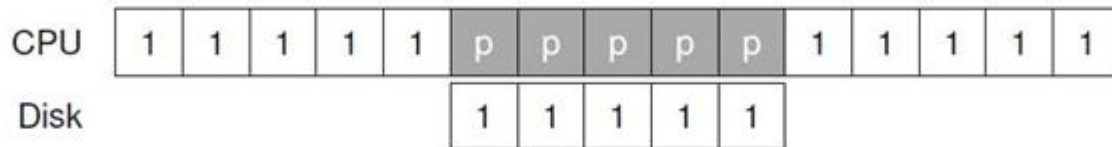
1. Loop while STATUS == BUSY (Wait)
2. Write to DATA register
3. Write to COMMAND register
4. Loop while STATUS == BUSY (Wait)



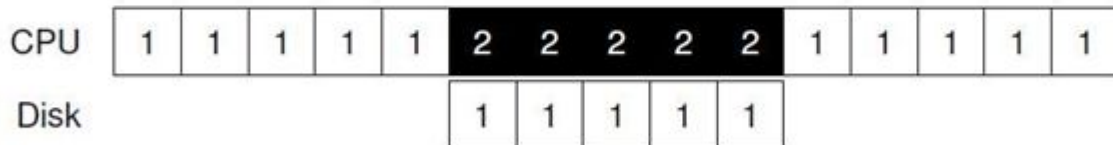
- One simple means of device **handshaking** involves polling:
 - The host repeatedly checks the **busy bit** on the device until it becomes clear.
 - The host writes a byte of data into the data-out register, and sets the **write bit** in the command register (in either order.)
 - The host sets the **command ready bit** in the command register to notify the device of the pending command.
 - When the device controller sees the command-ready bit set, it first sets the busy bit.
 - Then the device controller reads the command register, sees the write bit set, reads the byte of data from the data-out register, and outputs the byte of data.
 - The device controller then clears the **error bit** in the status register, the command-ready bit, and finally clears the busy bit, signaling the completion of the operation.
- Polling can be very fast and efficient, if both the device and the controller are fast and if there is significant data to transfer. It becomes inefficient, however, if the host must wait a long time in the busy loop waiting for the device, or if frequent checks need to be made for data that is infrequently there.

2. Interrupts

- Polling wastes CPU cycles



- Instead, OS can put process to sleep and switch to another process



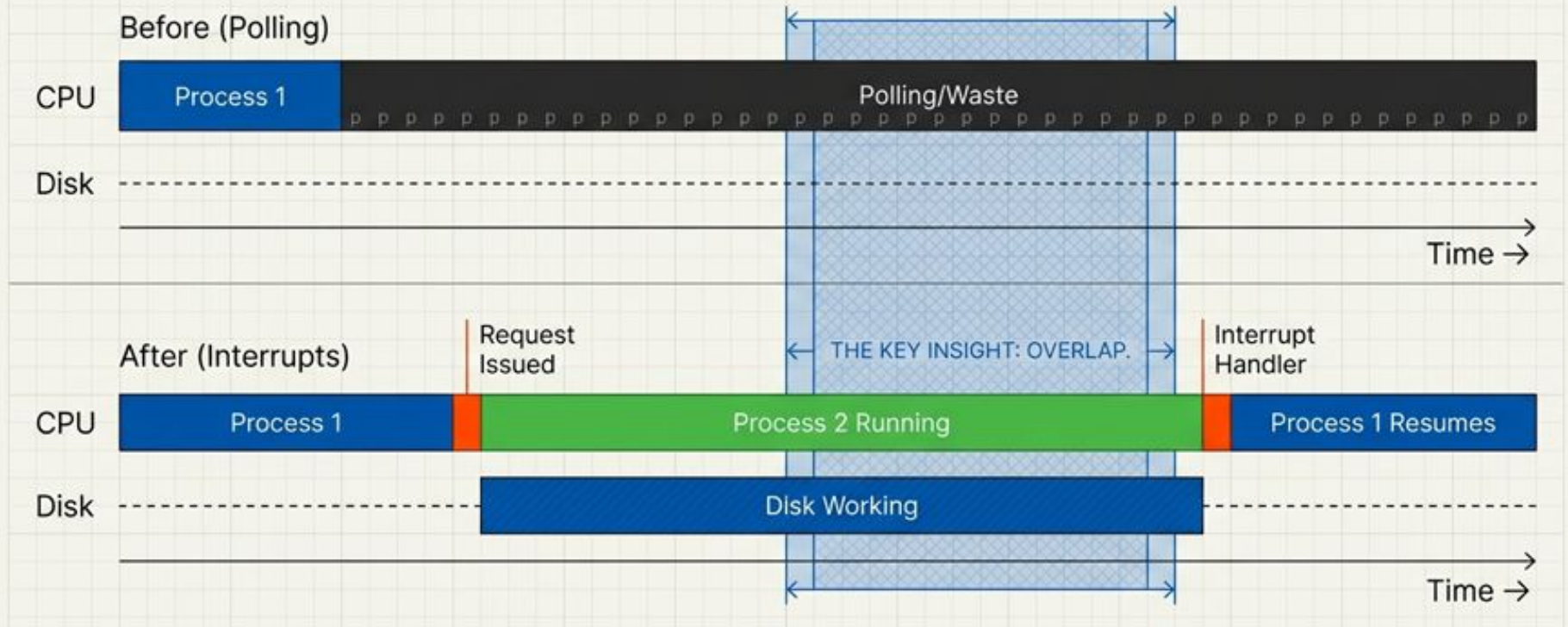
- When I/O request completes, device raises interrupt, OS can switch back to original process after that request has completed
 - Note: context switch to original process need not be immediate

- In the diagram 1 , Process 1 runs on the CPU for sometime(indicated by a repeated 1 on the CPU line), and then issues an I/O request to the disk to read some data.
- Without interrupts, the system simply spins, polling the status of the device repeatedly until the I/O is complete (indicated by a **p**).
- The disk services the request and finally Process 1 can run again.
- If instead we utilize interrupts and allow for overlap, the OS can do something else while waiting for the disk.
- In this diagram 2, the OS runs Process 2 on the CPU while the disk services Process 1's request. When the disk request is finished, an interrupt occurs, and the OS wakes up Process 1 and runs it again. Thus, both the CPU and the disk are properly utilized during the middle stretch of time.

- Instead of polling the device repeatedly, the OS can issue a request, put the calling process to sleep, and context switch to another task. When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a predetermined interrupt service routine (ISR) or more simply an interrupt handler.
- The handler is just a piece of operating system code that will finish the request (for example, by reading data and perhaps an error code from the device) and wake the process waiting for the I/O, which can then proceed as desired. Interrupts thus allow for overlap of computation and I/O, which is key for improved utilization.

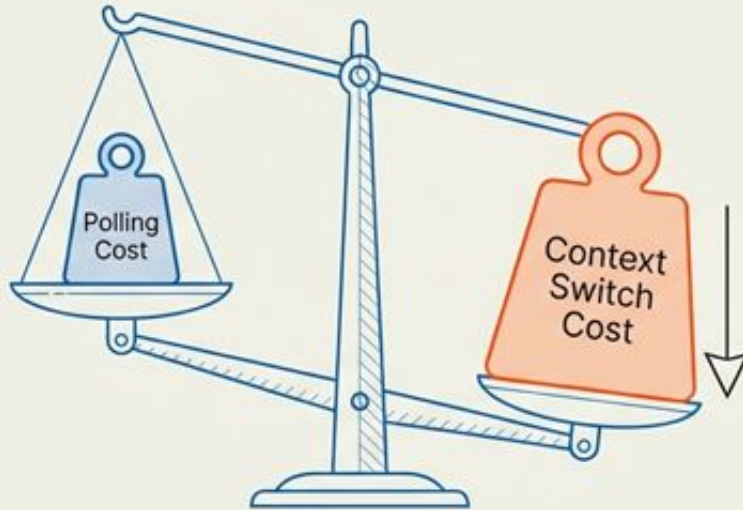
The Optimization: Interrupts

The Hero: Overlap allows the CPU to sleep.



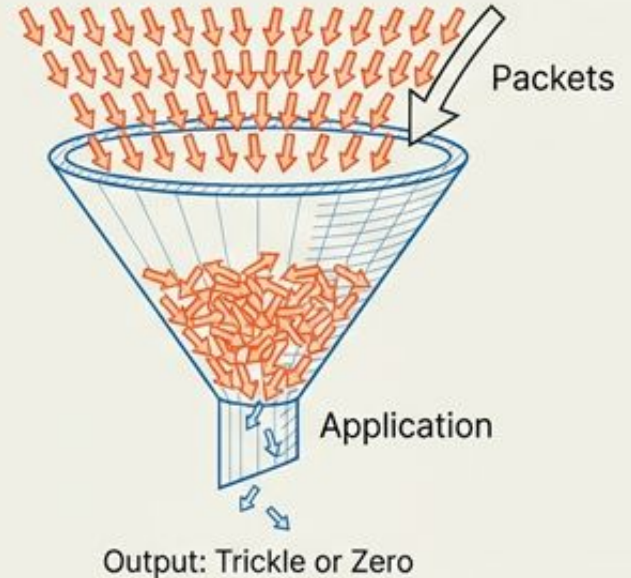
When Interrupts Fail

Scenario A: The Fast Device



Rule: If the device is faster than the time it takes to put a process to sleep, just poll. Don't sleep.

Scenario B: Livelock (The Network Flood)



Livelock: If every packet triggers an interrupt, the CPU spends 100% of its time handling interrupts and 0% running the application. The system is alive, but making no progress.

Interrupt handler

- Interrupt from I/O device causes trap, switches process to kernel mode
- Interrupt Descriptor Table (IDT) stores pointers (value of PC) to OS interrupt handlers (interrupt service routines)
 - Interrupt (IRQ) number identifies the interrupt handler to run for a device
- Interrupt handler processes notification from device, unblocks the process waiting for I/O (if any), and starts next I/O request (if any pending)
- Handling interrupts imposes kernel mode transition overheads
 - Note: polling may be faster than interrupts if device is fast

The CPU as a Manual Laborer

The diagram illustrates the CPU acting as a manual laborer, moving data between Main Memory and a Device. The CPU is shown in the center, connected to both components via thick blue arrows indicating data flow. The Main Memory is on the left, and the Device is on the right. The CPU is depicted as a square chip with pins. Below the CPU, a horizontal bar labeled "CPU Track" shows a sequence of red squares, each containing a small white 'c'. An arrow points from the text "CPU Utilization: 100% occupied with trivial data movement" to the end of the CPU Track.

Main Memory

CPU

Device

Load... Store...
Load... Store...
Load... Store...

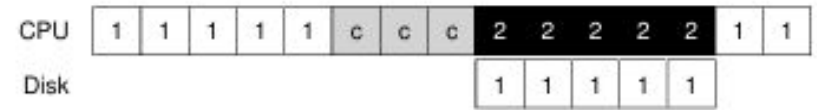
Load... Store...
Load... Store...
Load... Store...

CPU Track

CPU Utilization: 100% occupied with trivial data movement

Sajana.A,TKMIT, Kollam

- when using programmed I/O (PIO) to transfer a large chunk of data to a device, the CPU is once again over burdened with a rather trivial task, and thus wastes a lot of time and effort that could better be spent running other processes.

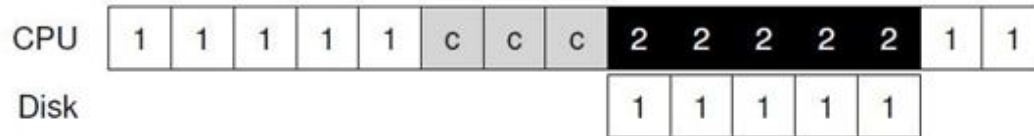


- In the timeline, Process 1 is running and then wishes to write some data to the disk. It then initiates the I/O, which must copy the data from memory to the device explicitly, one word at a time (marked c in the diagram). When the copy is complete, the I/O begins on the disk and the CPU can finally be used for something else.

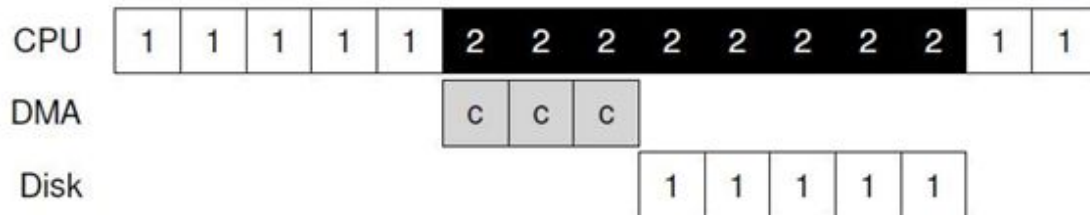
With PIO, the CPU spends too much time moving data to and from devices by hand. How can we offload this work and thus allow the CPU to be more effectively utilized?

4. Direct Memory Access (DMA)

- In spite of interrupts, CPU cycles wasted in copying data to/from device



- Instead, a special piece of hardware (DMA engine) copies from main memory to device and vice versa, without involving CPU
 - CPU gives DMA engine the memory location of data
 - In case of disk read, device copies data via DMA to RAM and then raises interrupt
 - In case of write, device copies data via DMA from RAM and then starts writing



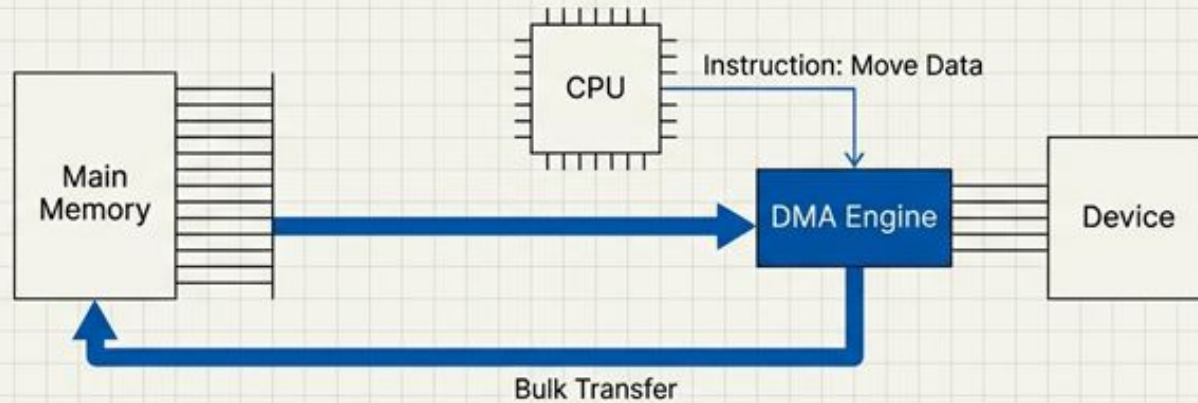
- A DMA engine is essentially a very specific device within a system that can orchestrate transfers between devices and main memory without much CPU intervention.
- DMA works as follows.
 - To transfer data to the device, the OS would program the DMA engine by telling it where the data lives in memory, how much data to copy, and which device to send it to.
 - At that point, the OS is done with the transfer and can proceed with other work.
 - When The DMA is complete, the DMA controller raises an interrupt, and the OS thus knows the transfer is complete.
- From the Timeline, you can see that the copying of data is now handled by the DMA controller. Because the CPU is free during that time, the OS can do something else, here choosing to run Process 2. Process 2 thus gets to use more CPU before Process 1 runs again.

Summary of disk read with interrupt + DMA

- Process P1 makes read system call to read data from disk
- OS gives command to disk via command register
- OS switches to another process P2 (P1 cannot run anymore, blocked)
- Disk completes reading data, copies data into main memory directly via DMA, then raises interrupt
- OS handles interrupts, disk data is ready, marks P1 as ready to run
 - Interrupt handled in the kernel mode of P2
- OS scheduler switches back to P1 at a later time

The Solution: Direct Memory Access (DMA)

Delegating the Heavy Lifting



The "copy" operation disappears from the CPU line. The CPU is free.

5. Device interaction methods

Method 1: I/O Instructions



The Old Way. Explicit, privileged instructions.

```
outb(port, data);
```

Dialing a specific, private phone number.

Method 2: Memory-Mapped I/O



The Modern Way. Device registers look like memory addresses.

```
*device_ptr = data;
```

Leaving a note in a specific mailbox that the hardware checks.

Both methods require Privileged Mode (OS only) to prevent chaos.

1. I/O instructions

- a. oldest method (used by IBM mainframes for many years) is to have explicit I/O instructions.
- b. These instructions specify a way for the OS to send data to specific device registers and thus allow the construction of the protocols
- c. For example, to send data to a device, the caller specifies a register with the data in it, and a specific port which names the device. Executing the instruction leads to the desired behavior.
- d. Such instructions are usually privileged. The OS controls devices, and the OS thus is the only entity allowed to directly communicate with them.

2. Memory Mapped I/O

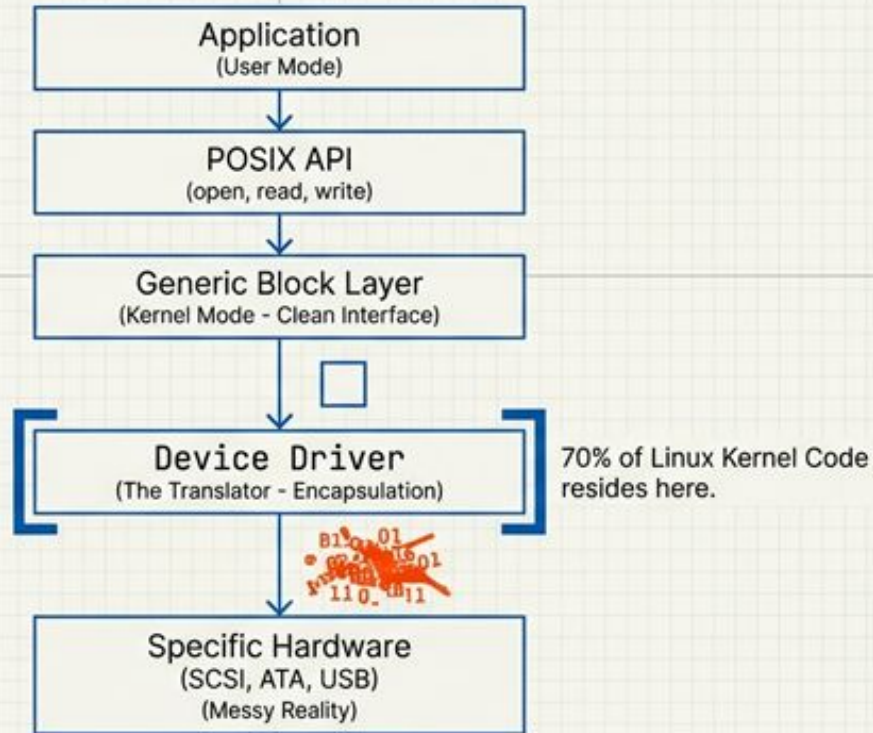
- With this approach, the hardware makes device registers available as if they were memory locations.
- To access a particular register, the OS issues a load (to read) or store (to write) the address; the hardware then routes the load/store to the device instead of main memory.

How does OS read/write to device registers?

- OS communicates with device via **device controller** on the device
- OS code that communicates with device is called **device driver**
- OS reads/writes registers in the I/O device: how?
- **Explicit I/O instructions**
 - E.g., on x86, `in` and `out` instructions can be used to read and write to specific registers on a device
 - Privileged instructions accessed by OS
- **Memory mapped I/O**
 - Device makes registers appear like memory locations
 - OS simply reads and writes from memory
 - Memory hardware routes accesses to these special memory addresses to devices

6. Device Drivers

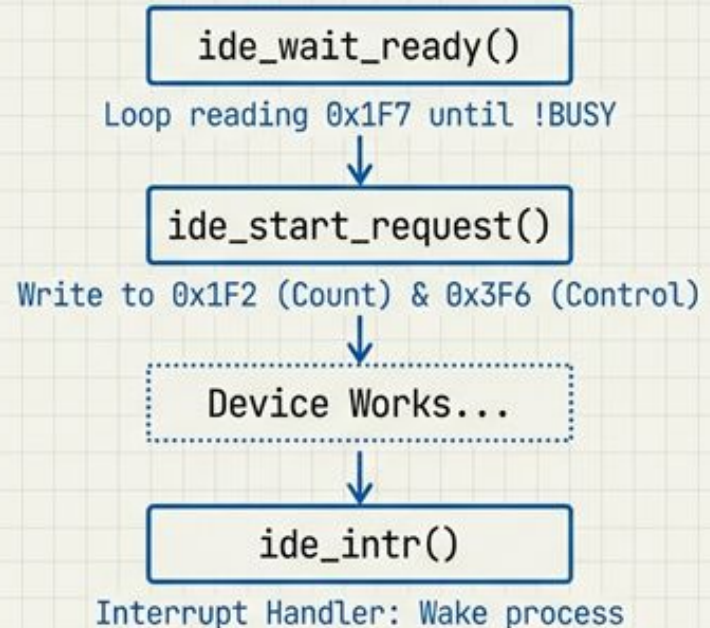
The Crux: How to build a device-neutral OS?



Case Study: The xv6 IDE Driver



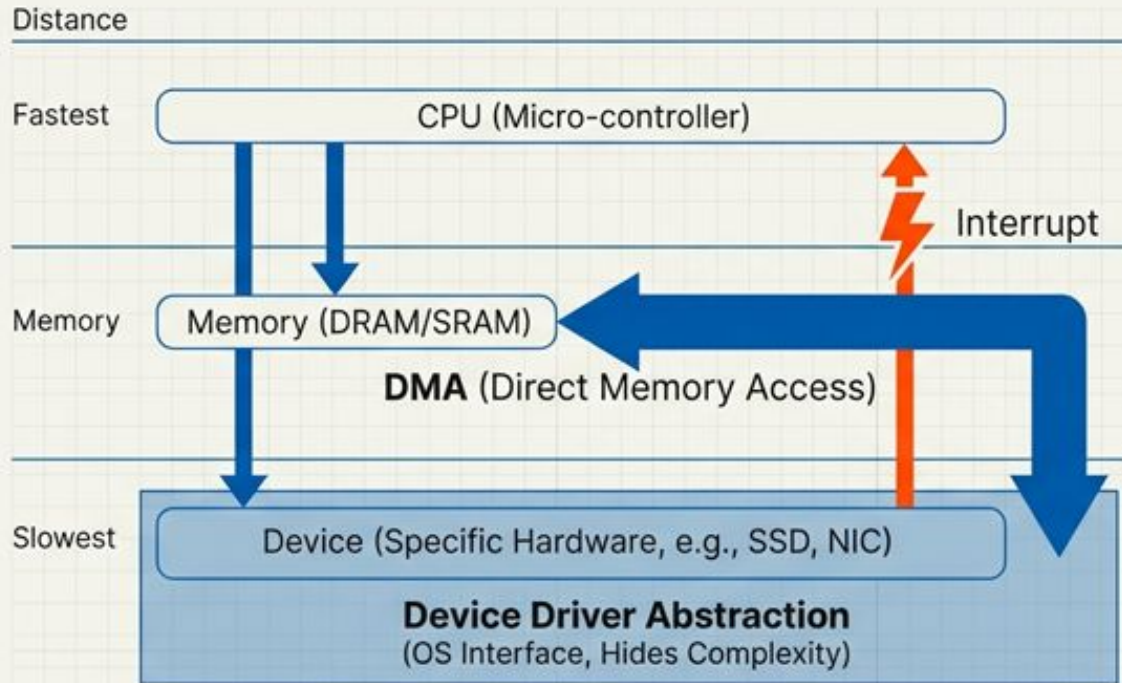
Micro-controller (CPU)
Memory (DRAM or SRAM or both)
Other Hardware-specific Chips



Reality Check: Even with interrupts, the code must briefly poll "Wait Ready" to ensure the hardware is listening.

The Modern I/O Stack

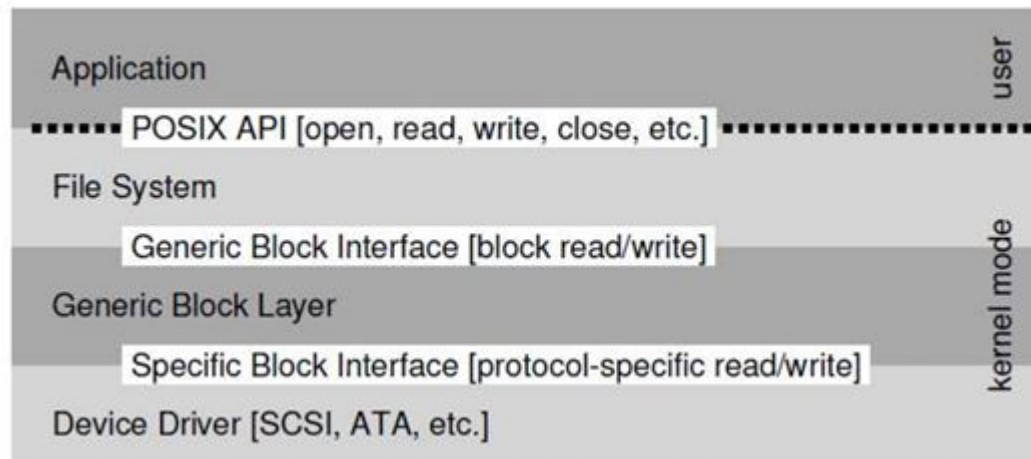
Orchestrating the gap between blazing speed and physical latency.



	SUMMARY OF I/O ARCHITECTURE
	• 1. Hierarchy manages Cost. • 2. Interrupts manage Latency. • 3. DMA manages Throughput. • 4. Drivers manage Compatibility.

I/O stack in the kernel

- Device driver: part of OS code that talks to specific device, gives commands, handles interrupts etc.
 - Rest of OS code abstracts out the device-specific details
- I/O subsystems (file system / networking) built as layers: system calls, block read/write, device drivers that communicates with I/O device

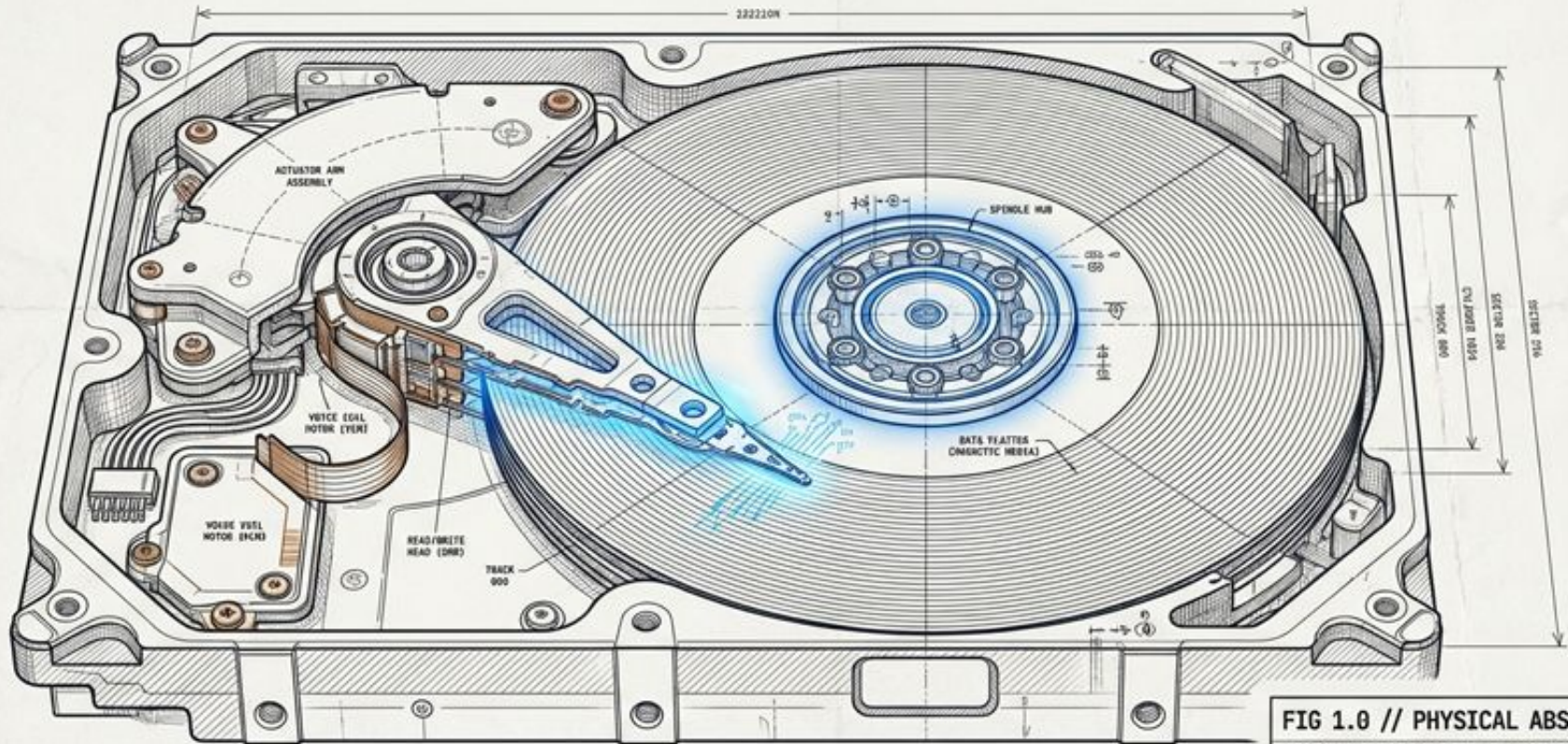


- The diagram also shows a raw interface to devices, which enables special applications (such as a file-system checker, or a disk defragmentation tool) to directly read and write blocks without using the file abstraction.
- Most systems provide this type of interface to support these low-level storage management applications.

Hard Disk

Hard Disk Drives: The Mechanical Heart of Storage

Geometry, Latency, and the Art of Disk Scheduling

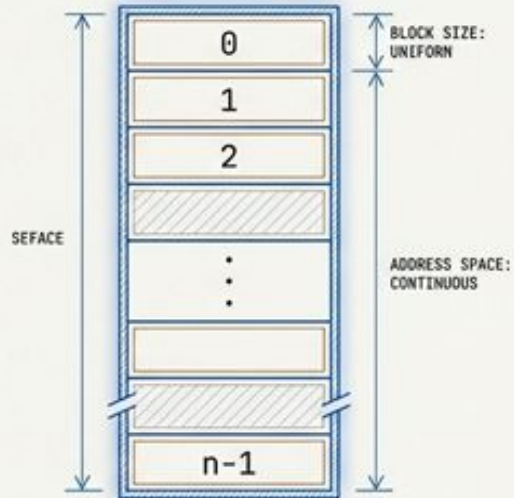


Hard disk drives

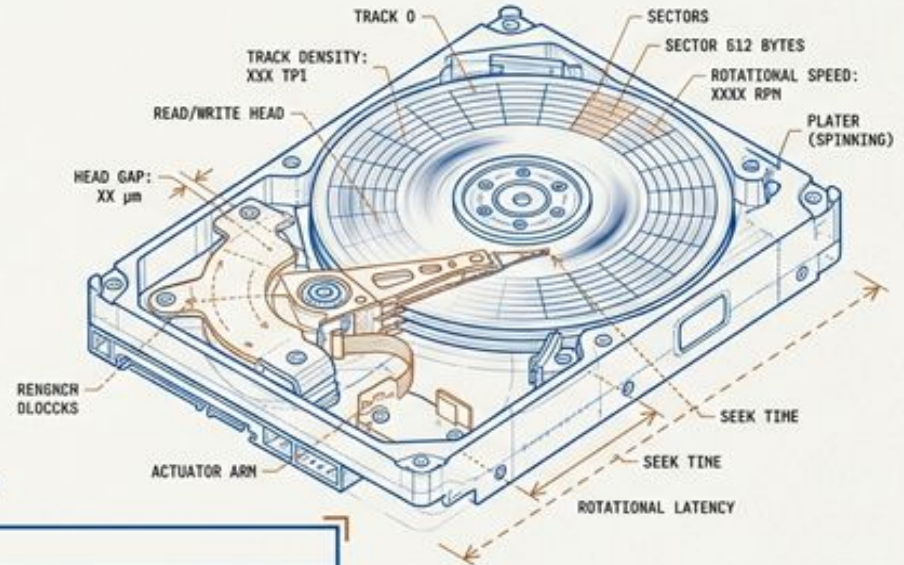
- These drives have been the main form of persistent data storage in computer systems for decades and much of the development of file system technology (coming soon) is predicated on their behavior.
- Here we study
 - How do modern hard-disk drives store data?
 - What is the interface?
 - How is the data actually laid out and accessed?
 - How does disk scheduling improve performance

The Interface is an Illusion

LOGICAL VIEW (OS)



PHYSICAL REALITY (HDD)



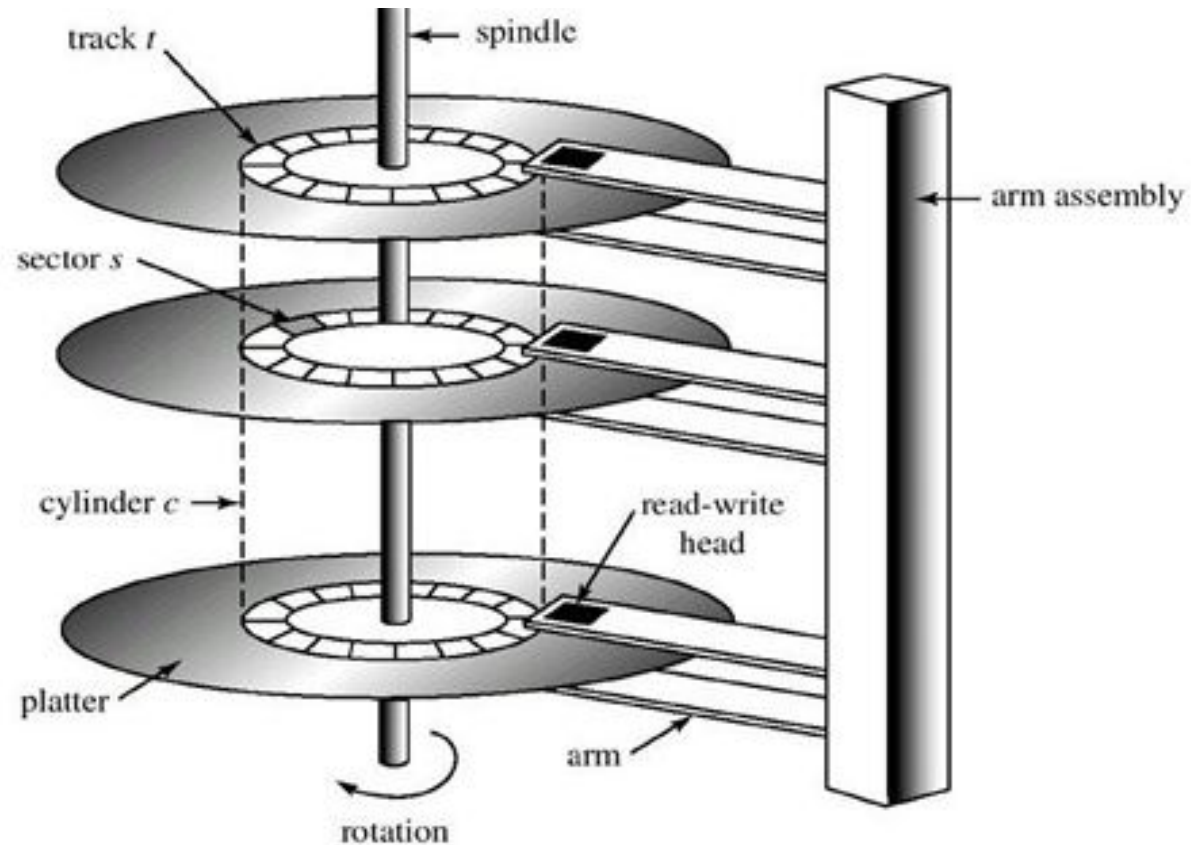
NOTE

The Unwritten Contract:

Accessing contiguous blocks (Sequential) is fast. →

Jumping between blocks (Random) is slow. ↘

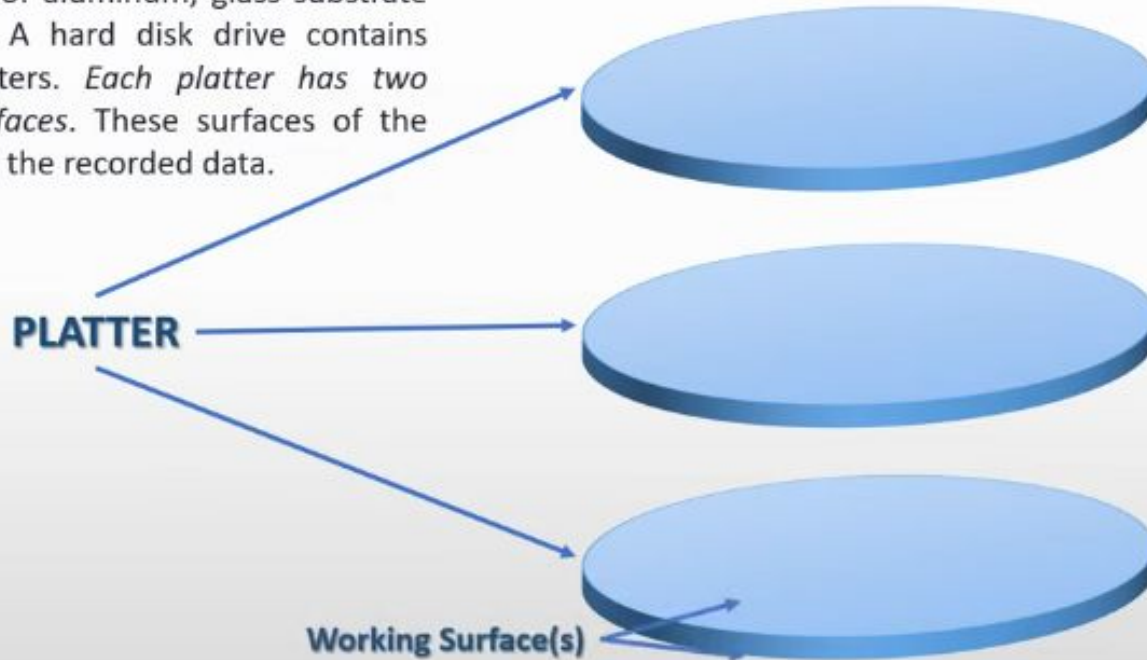
The only guarantee is a 512-byte atomic write.



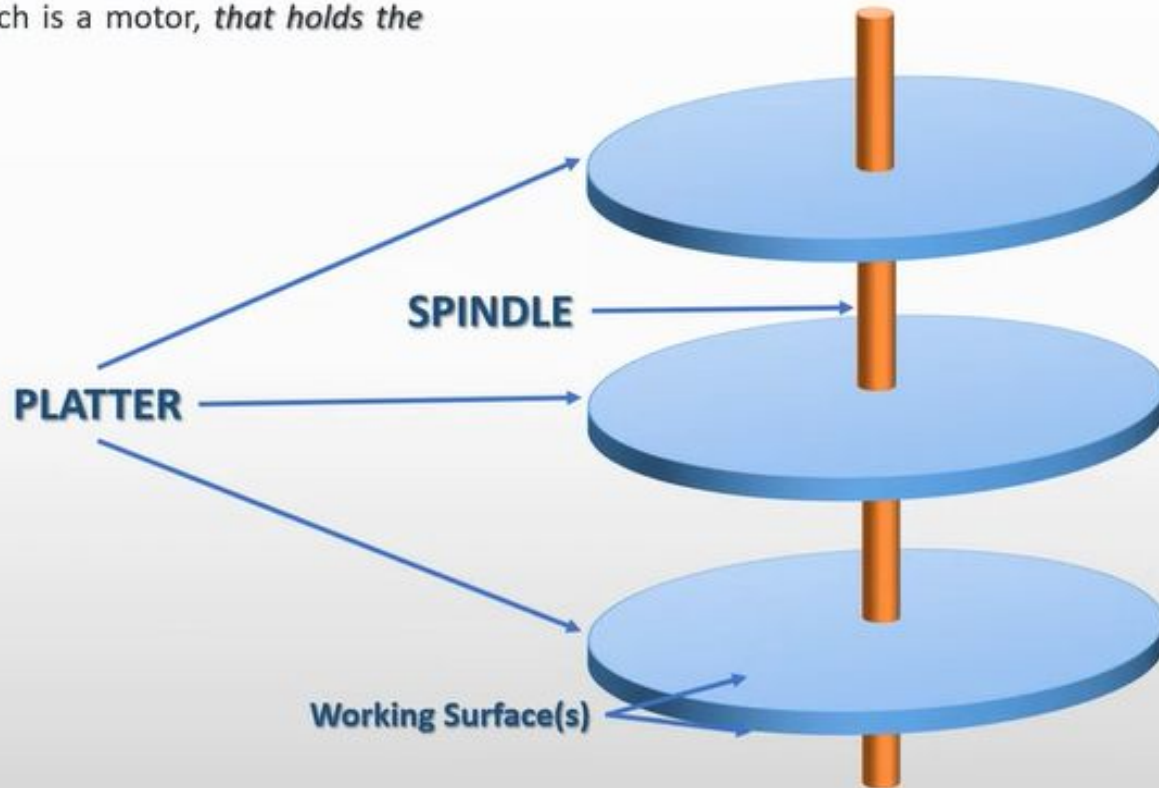
(Structure of Magnetic disks (Harddisk drive))

STRUCTURE OF HARD DISK DRIVE (HDD) – SECONDARY MEMORY

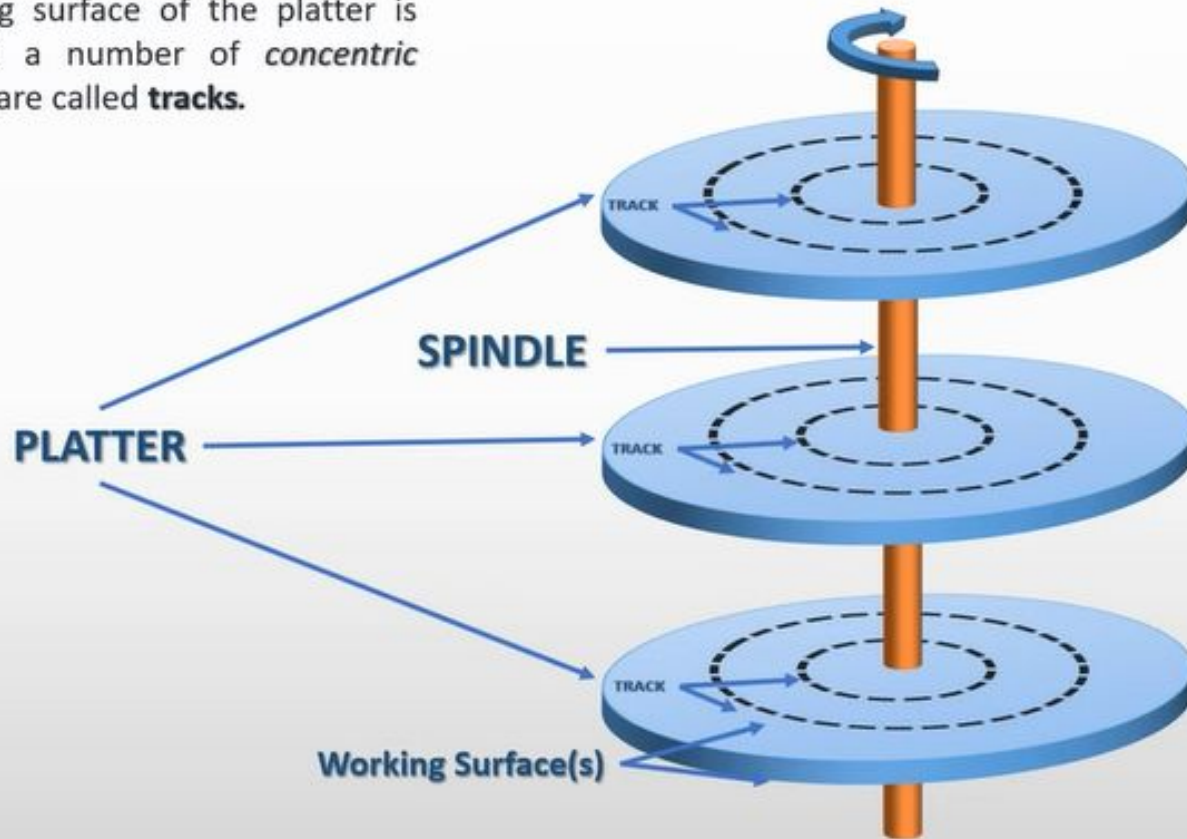
A **platter** is a *circular magnetic plate* that is used for *storing data* in a hard disk. It is often made of aluminum, glass substrate or ceramic. A hard disk drive contains several platters. *Each platter has two working surfaces*. These surfaces of the platters hold the recorded data.



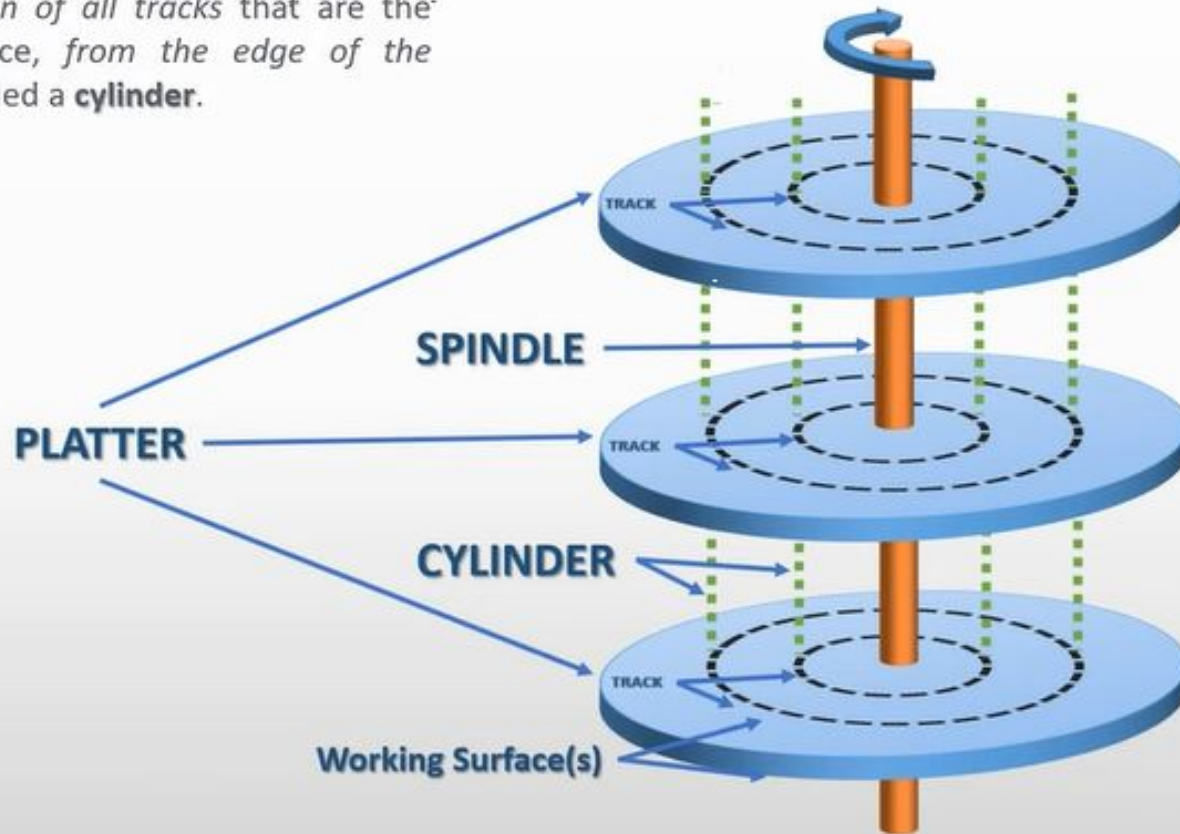
A typical HDD design consists of a **spindle**, which is a motor, *that holds the platters*.



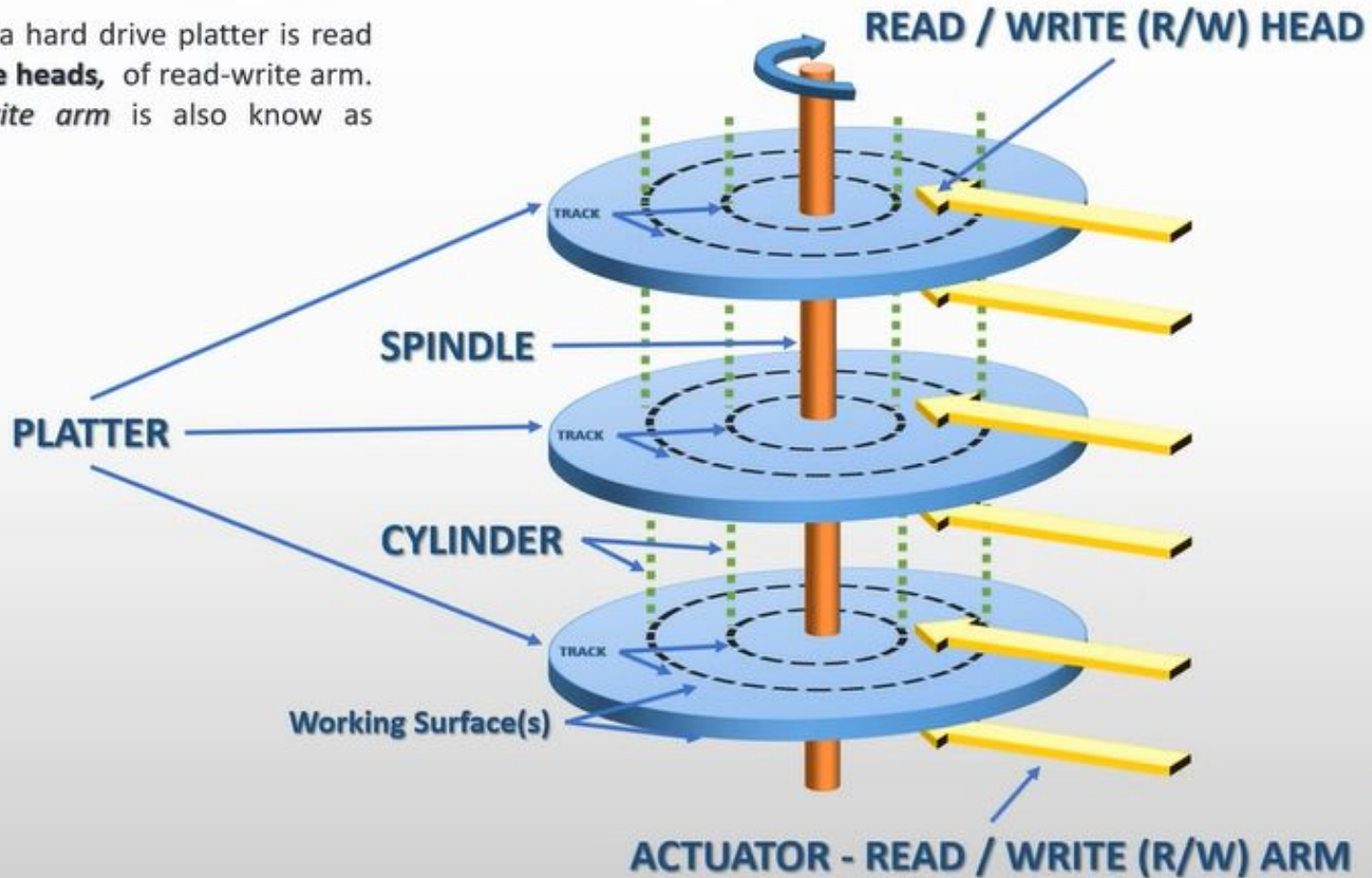
Each working surface of the platter is divided into a number of *concentric rings*, which are called **tracks**.



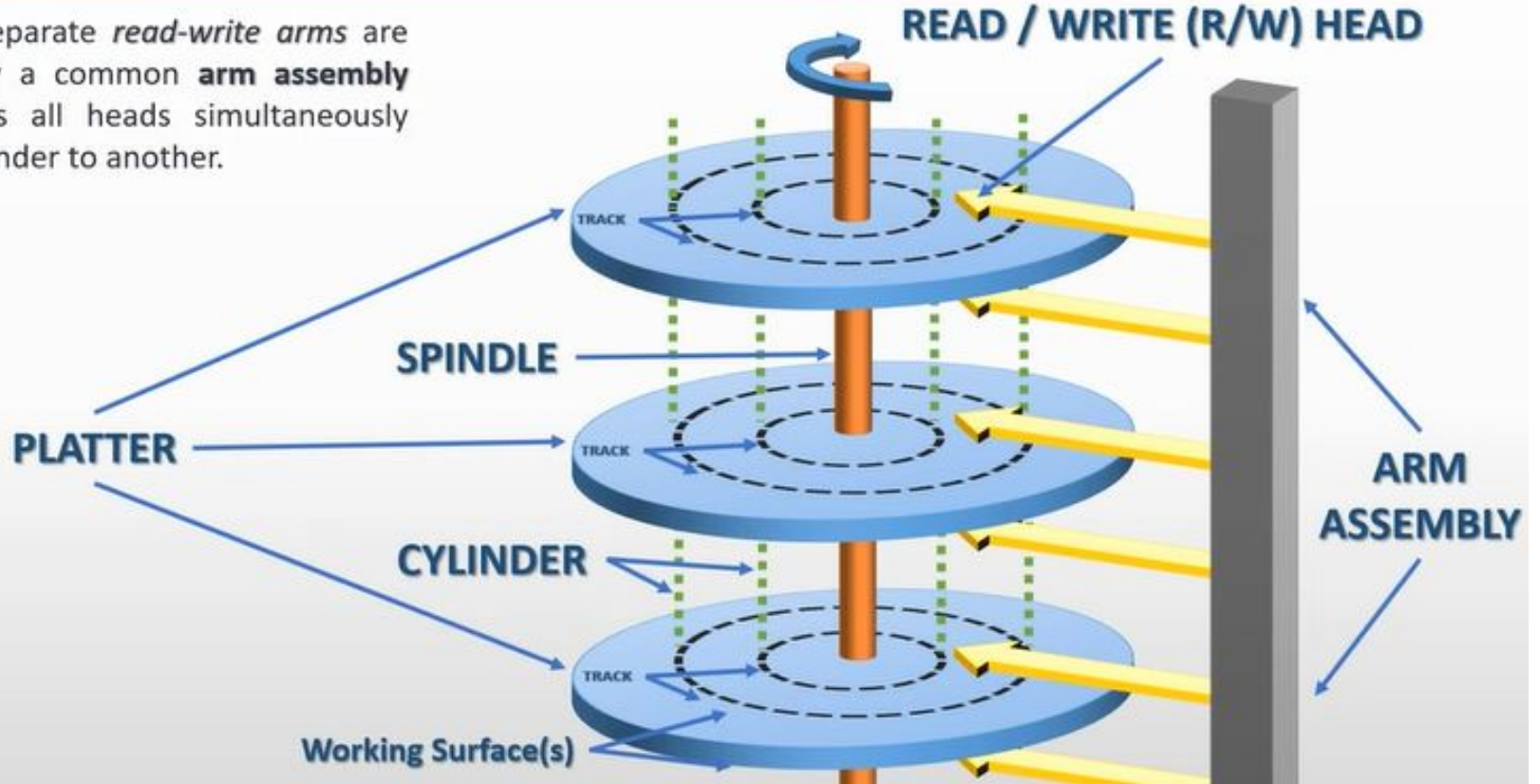
The *collection of all tracks* that are the same distance, *from the edge of the platter*, is called a **cylinder**.



The data on a hard drive platter is read by **read-write heads**, of read-write arm. The *read-write arm* is also known as **actuator**.

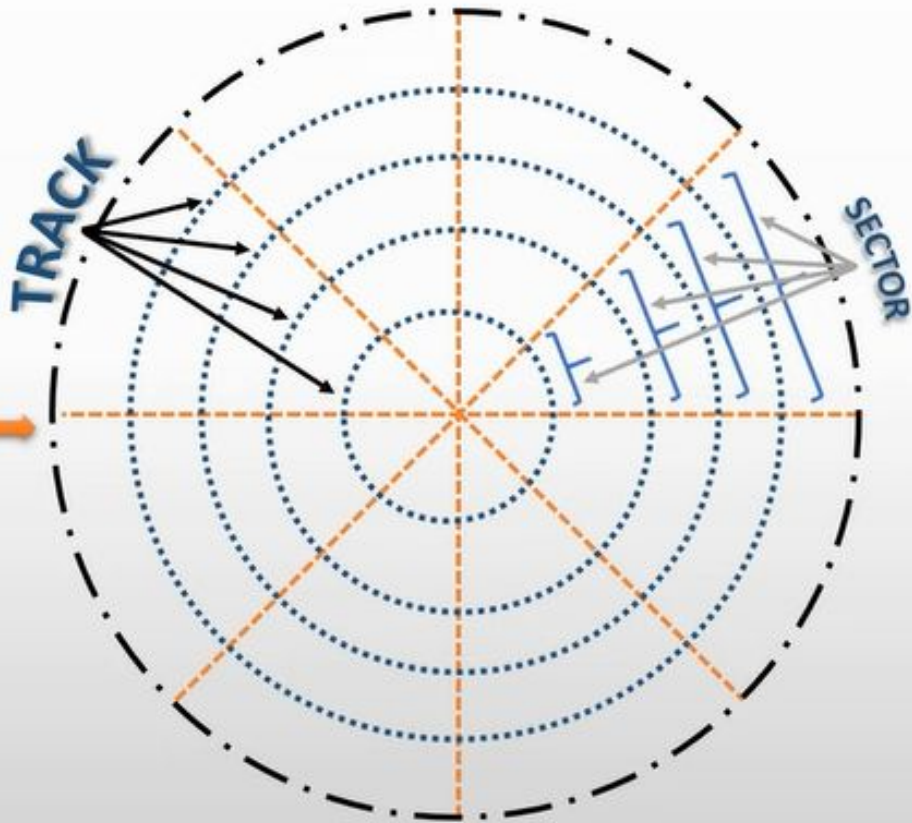


Each on a separate *read-write arms* are *controlled* by a common **arm assembly** which moves all heads simultaneously from one cylinder to another.



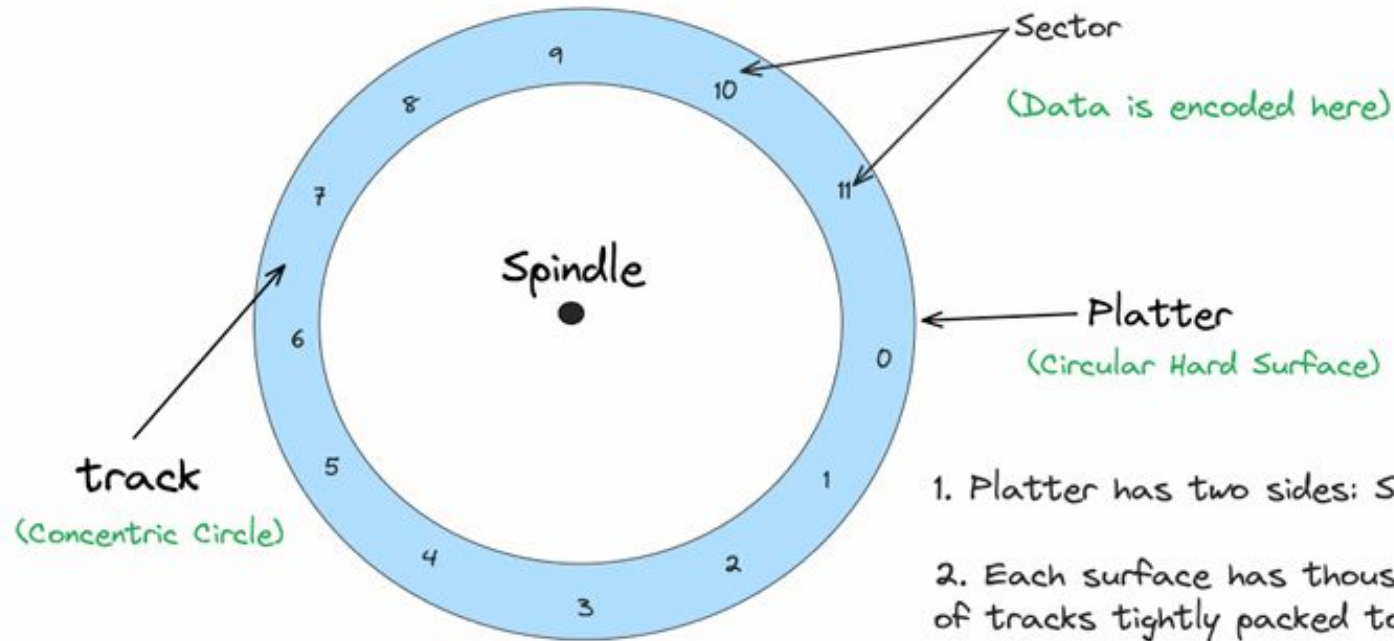
SECTORS: Each track is further broken down into smaller units called sectors. As sector is the basic unit of data storage on a hard disk, each track has the same number of sectors, which means that the sectors are packed much closer together on tracks near the center of the disk. A single track typically can have thousands of sectors. The data size of a sector is always a power of two, and is almost always either 512 or 4096 bytes.

PLATTER



- The drive consists of a large number of sectors (512-byte blocks), each of which can be read or written.
- The sectors are numbered from 0 to $n - 1$ on a disk with n sectors.
- Thus, we can view the disk as an array of sectors; 0 to $n - 1$ is thus the address space of the drive.
- Hard disk drive (HDD) geometry refers to the **physical, hierarchical** organization of data on spinning magnetic platters, defined by **cylinders, heads, and sectors (CHS)**.
- Data is stored on both sides of circular, metal/glass platters (surfaces/heads) in concentric rings called tracks, which are divided into smaller, 512-byte to 4KB storage units called sectors

Basic Geometry

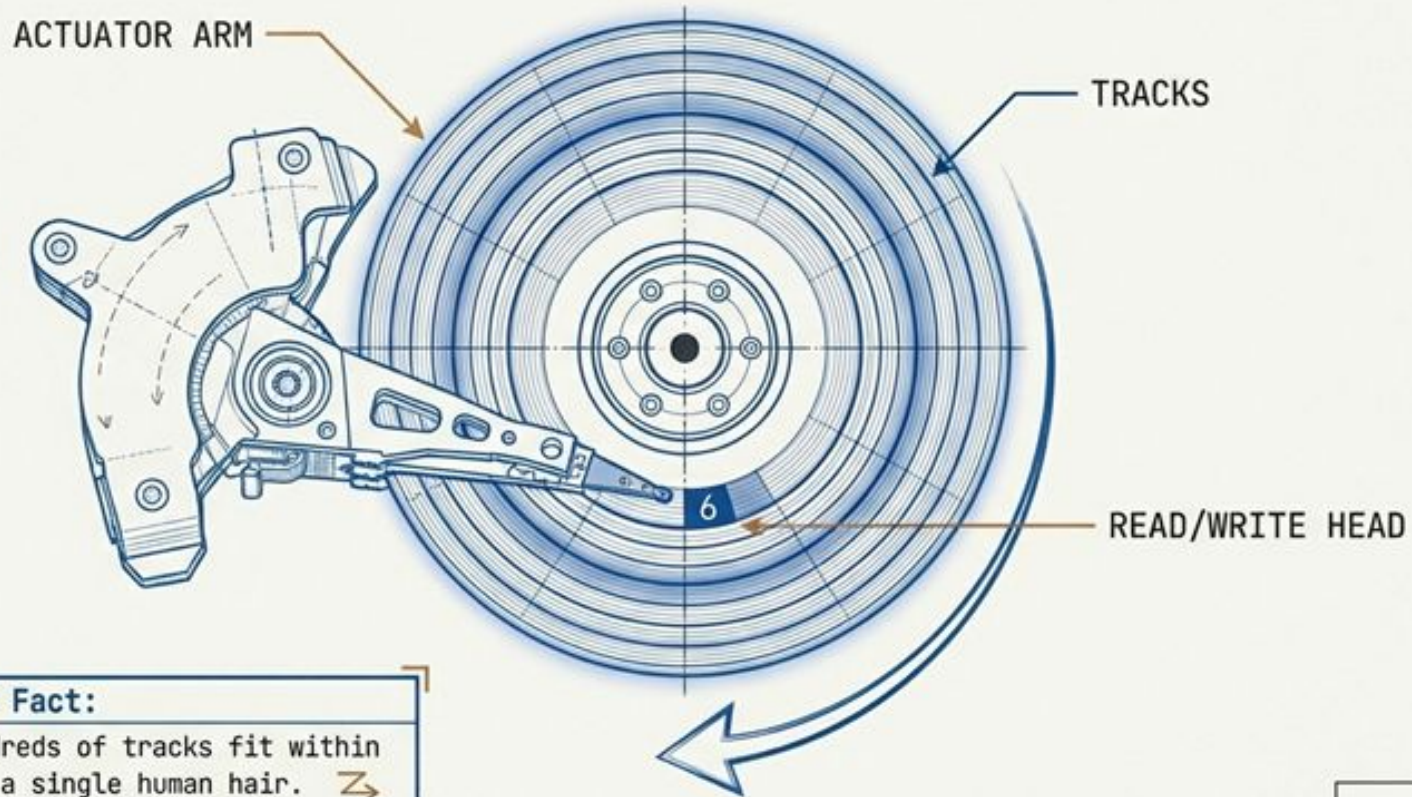


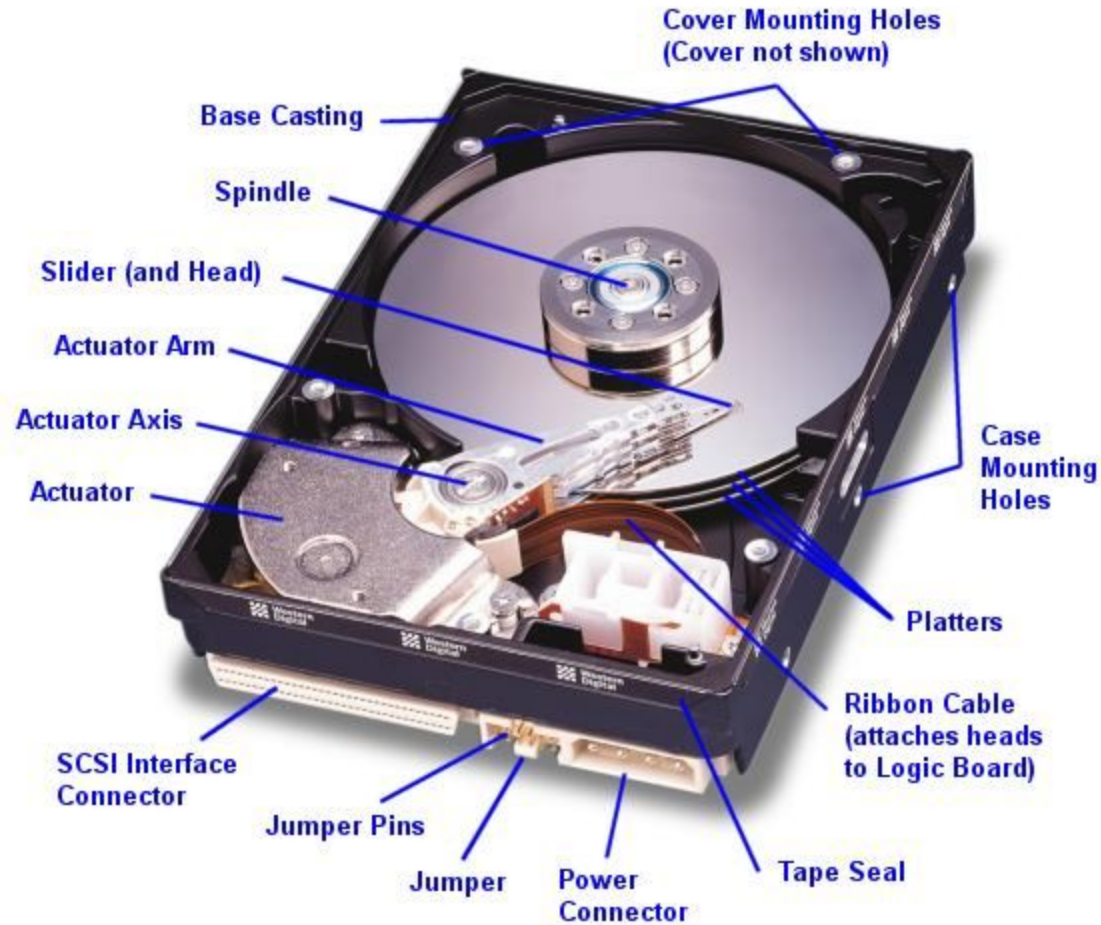
Spindle is connected to a motor => spins the platter around

1. Platter has two sides: Surface
2. Each surface has thousands of tracks tightly packed together
3. Each track is just having width of human hair

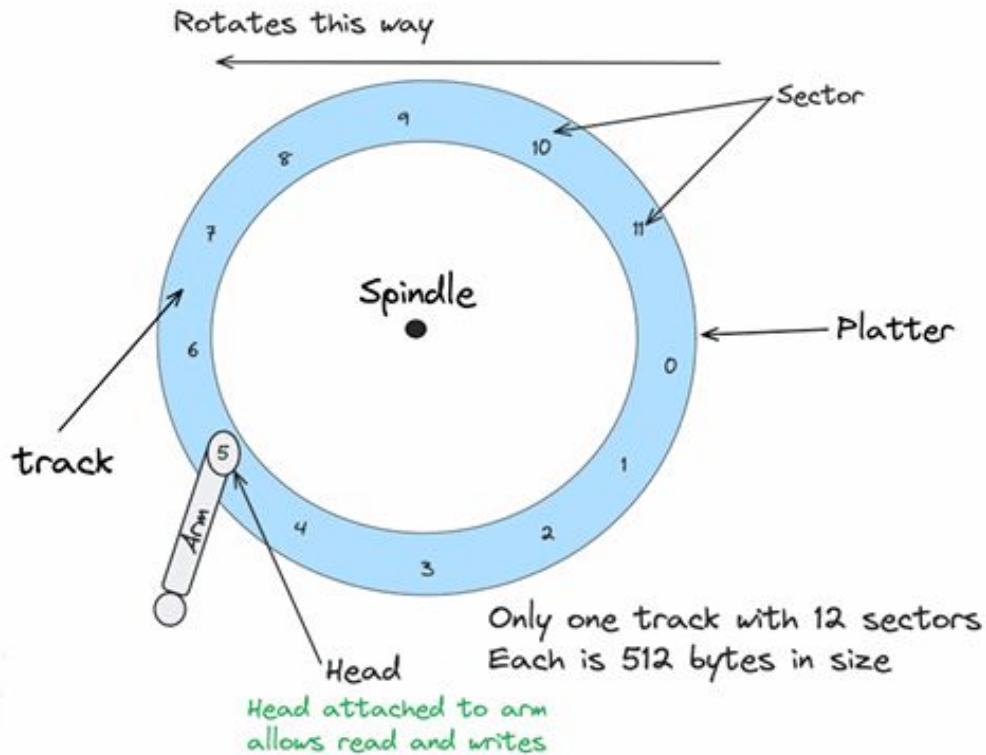
- **Platter**, a circular hard surface on which data is stored persistently by inducing magnetic changes to it.
- A disk may have one or more platters; each platter has 2 sides, each of which is called a **surface**.
- The platters are all bound together around the **spindle**, which is connected to a motor that spins the platters around (while the drive is powered on) at a constant (fixed) rate.
- Data is encoded on each surface in concentric circles of sectors; we call one such concentric circle a **track**.
- **To read and write from the surface, we need a mechanism that allows us to either sense (i.e., read) the magnetic patterns on the disk or to induce a change in (i.e., write) them.**
- **This process of reading and writing is accomplished by the disk head; there is one such head per surface of the drive. The disk head is attached to a single disk arm, which moves across the surface to position the head over the desired track.**

The Arm, The Head, and The Track





Simple Disk Drive



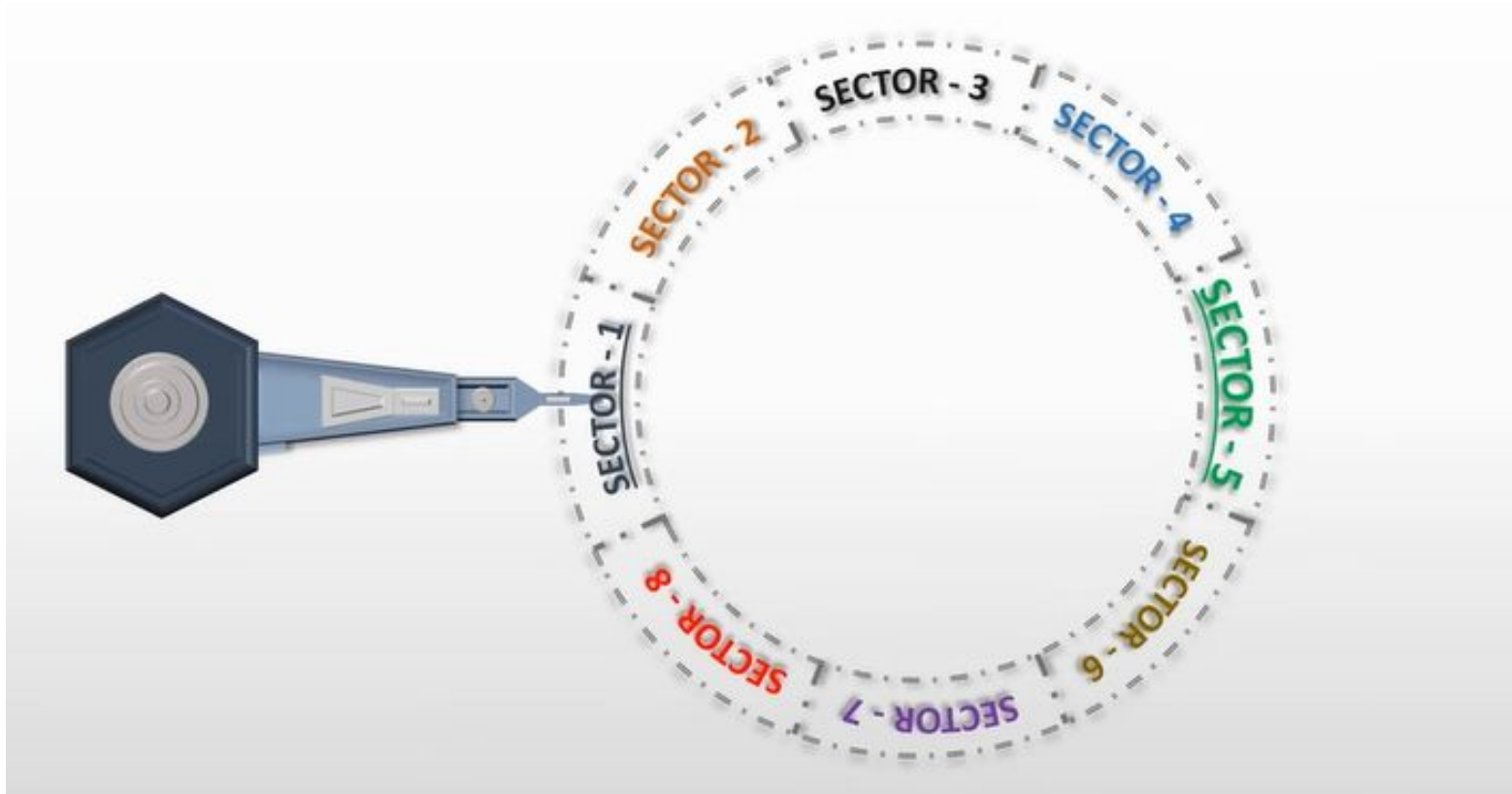
- Disk arm moves across to support reads and writes
- Spindle connected to a motor, spins the platter around at a constant fixed rate
- The rate is measured in rotations per minute (RPM)
- Modern values in range **7200 to 15000 RPM**
- For drive that rotates at 10,000 RPM, single rotations takes at **6ms**

Single-Track Latency: Rotational Delay

- Consider in the previous case that the access has to be done in block 0
 - Remember that the disk rotates and as soon as the head reaches the desired sector (block), it can read
 - The position of head was at sector 5
 - It has to wait for the disk to rotate and the head to reach at sector 0
 - This is known as **rotational delay** or **rotation delay**
 - What if the read access request arrived for sector 4?

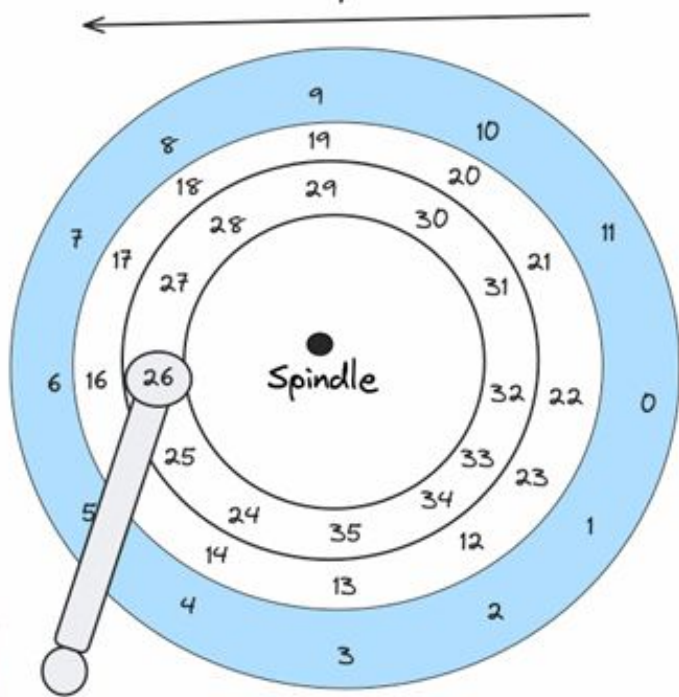
Almost full rotation!!





Multiple Tracks: Seek Time

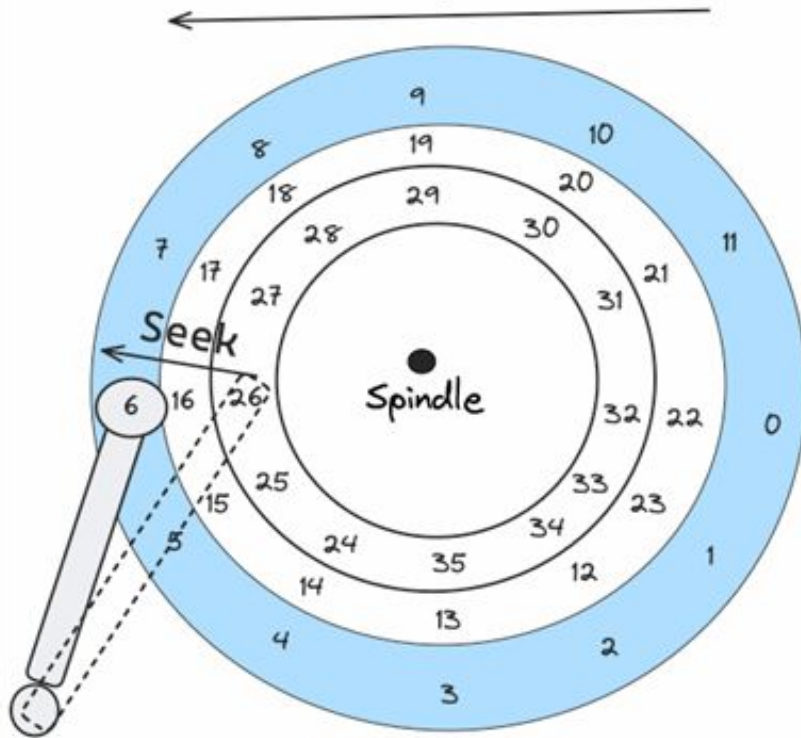
Rotates this way



- Its not just about rotational delay
- In a real setting, a disk surface has 100s of tracks
- Read or write may happen at any block located in different tracks
- Rotation will only help in movement within a track
- Across tracks, the operation performed is **Seek**

Multiple Tracks: Seek Time

Rotates this way



- Seek is a costly operation in terms of time
- Seek has multiple phases
 - **Acceleration:** Arm starts moving
 - **Coasting:** Arm moving at full speed
 - **Settling:** Head over correct track
- Settling time is 0.5 - 2ms High!
- Final phase of I/O is **transfer** - Read or write from surface
- **Seek, rotate and transfer** - three key phases



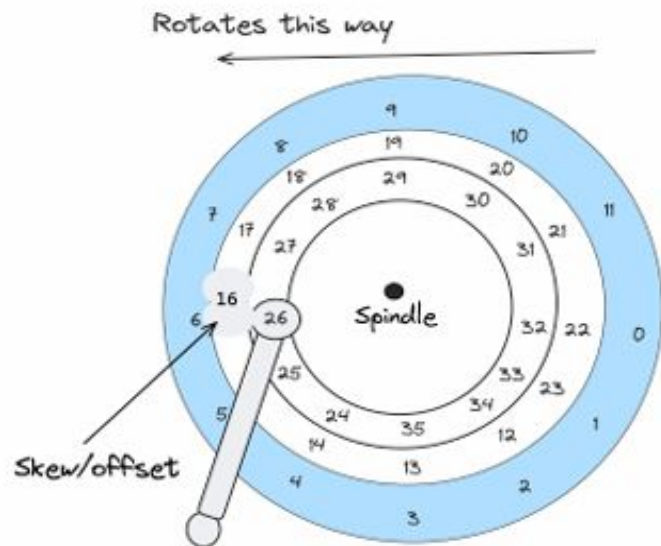
SEEK TIME

The time required by the read-write head assembly to move / reach / find the desire track / cylinder, i.e. move to one track / cylinder to another, is known as **Seek Time**.

ROTATIONAL LATENCY

The time required by the read-write head assembly to move / reach to the desire sector of the desire track, is known as **Rotational Latency**.

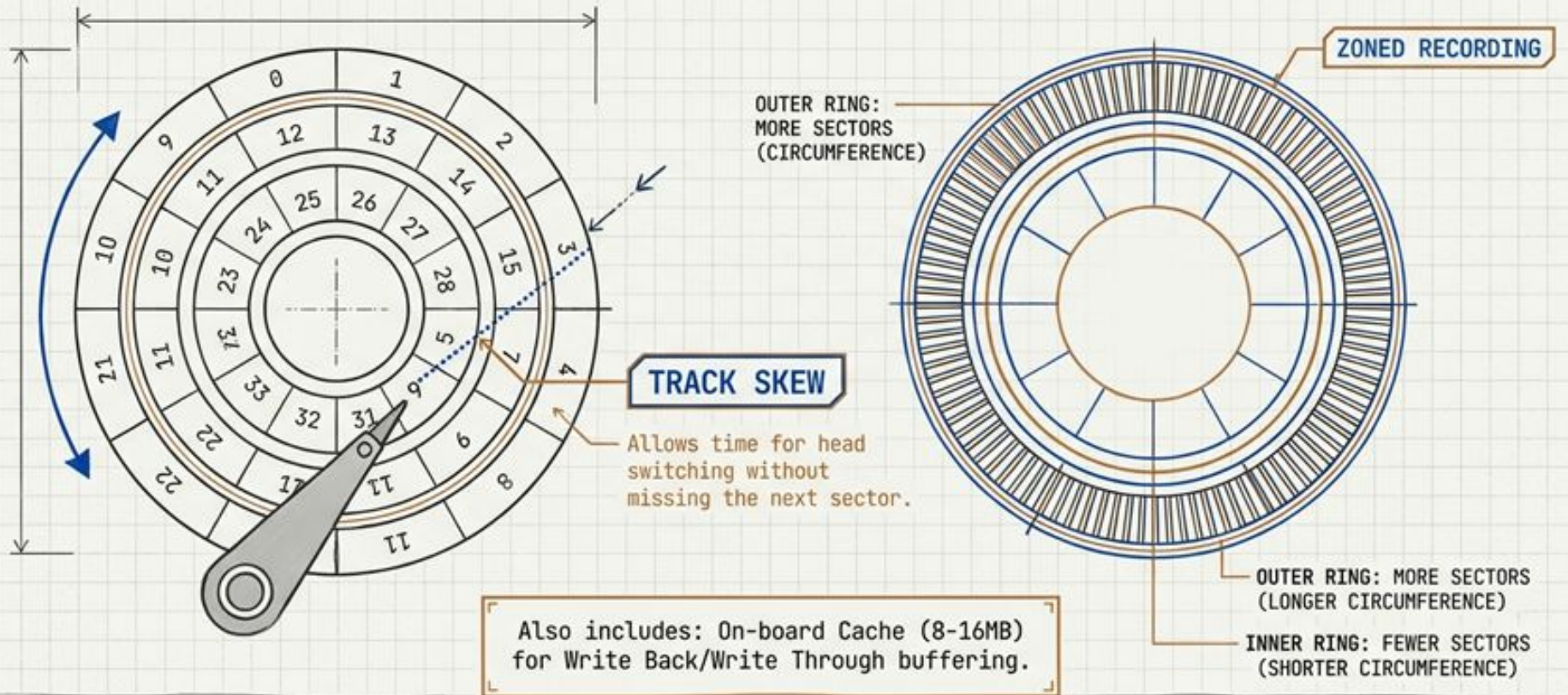
We may also have to consider skew



- Many drives some kind of **track skew** to make sure that sequential reads can be properly serviced
- When head moves from one track to another:
 - By the time the head moves, the desired block in the track would have got rotated
 - Head would now have to wait for a longer rotational delay
- To avoid this beginning of next track is slightly offset or **skewed**
- This is done to optimize performance



Engineering Reality Checks



$$\begin{aligned} &\text{ACCESS TIME} \\ &= \\ &\text{SEEK TIME + ROTATIONAL LATENCY} \end{aligned}$$

Modern Disk Drives also have Cache!

- The cache is often referred to as **Track buffer**
 - Allow the drive to quickly respond to requests
 - Small amount of memory (usually around 8 to 16 MB)
 - Can be used by drive to read from/write to the disk
- **Reads:** When reading from one sector, read all sectors in that track and cache
 - Subsequent reads can be very fast
- **Writes:** Two choices: **Write through** and **Write back**

Write on Cache

- **Writeback** (Immediate reporting)
 - Acknowledge a write has completed as soon as the data reaches cache memory
 - It makes drives appear very fast but it can be dangerous
 - Especially if order needs to be preserved - **This can lead to problems!**
- **Writethrough**
 - Acknowledge when the write has been written to the disk
 - Here data written to cache is also written to cache and disk simultaneously
 - Overall performance here might be an issue



Feature	Write-Through Cache	Write-Back (Write-Behind)
Write Destination	Cache + Main Memory simultaneously	Cache only (initially)
Speed	Slower (bottlenecked by memory speed)	Faster (writes to cache speed)
Consistency	High; data is always updated	Low; data may be inconsistent (dirty)
Data Safety	High; low risk of loss during crash	Lower; risk of data loss on power failure
Complexity	Simple	Complex (requires "dirty" bits)
Memory Traffic	High (every write goes to memory)	Low (only on eviction)
Use Case	Read-heavy, critical data	Write-heavy, high performance

Comparison
Table:
Write-Through
vs. Write-Back

Some Analysis

- Disk rotates at **10,000 RPM** and has transfer rate of **100 MB/sec**
 - **How much milliseconds does a single rotation take?**
 - 1 minute = 60 seconds = 60,000 ms
 - 10,000 RPM in 60,000 ms => **6 ms for 1 rotation**
 - **How much time to transfer 512 KB blocks of data?**
 - 0.5 MB of data
 - 100 MB/sec => 0.1 MB/ms => **5 ms for 0.5 MB of data**



I/O Time of Disk

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

$$\text{Rate of I/O} \quad R_{I/O} = \frac{Size_{transfer}}{T_{I/O}}$$

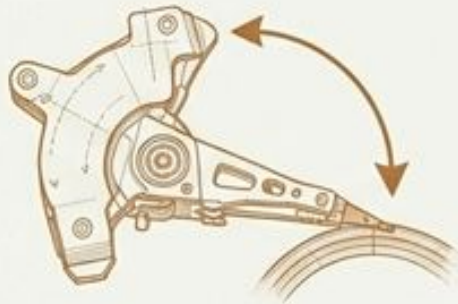
- We can perform different analysis given some characteristics
- Assume that there are two different workloads
 - Random workload
 - Sequential workload



The Anatomy of Latency

$$T_{I/O} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

SEEK



Time to move arm to the track.

ROTATION



Time to wait for sector
(Avg: $R/2$).

TRANSFER



Reading/Writing the bits.

I/O Time of Disks

- **Random Workload**

- Issues small (4 KB) reads to random locations on the disk
- Very common in applications like Database management systems

- **Sequential Workload**

- Reads large number of sectors consecutively from disk
- These are also quite common!

- Given workload, can we perform some comparison on the disk performance

- We would also need some disk characteristics



Disk Scheduling

- OS plays a role in the order of I/O issued to disk
- Given a set of I/O requests, is it possible to schedule them in an order
 - Disk scheduler decides which I/O request to schedule next!
 - Decides which request to schedule to improve performance
- How can disk scheduler make a guess on what request shall be better?
 - Remember: Disk scheduler can estimate how much time each request can take
 - Parameters like rotational delay, transfer, seek are known/easily estimated

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists
- In the past, operating system responsible for queue management, disk drive head scheduling
 - Several algorithms exist to schedule the servicing of disk I/O requests
 - The analysis is true for one or many platters
 - We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53

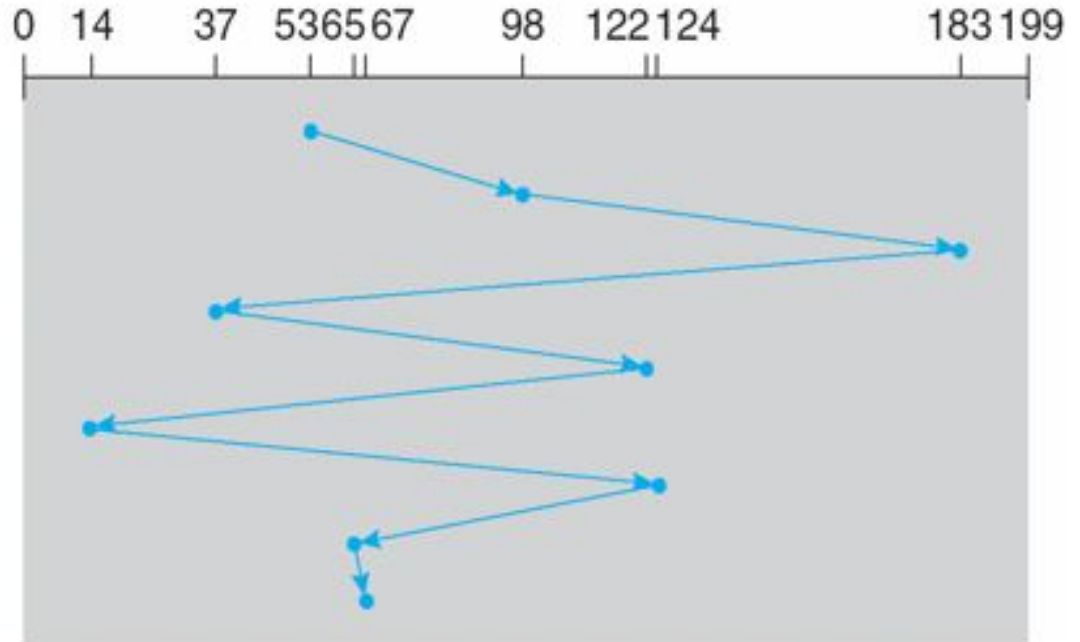
1. FCFS

- The simplest form of scheduling is first-in-first-out (FIFO) scheduling, which processes items from the queue in sequential order.

Illustration shows total head movement of 640 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

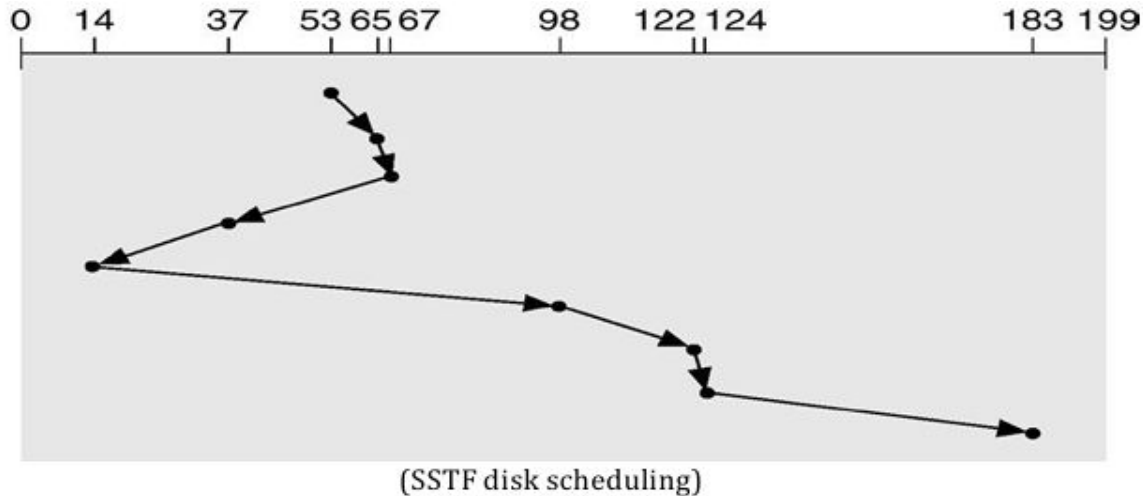
head starts at 53



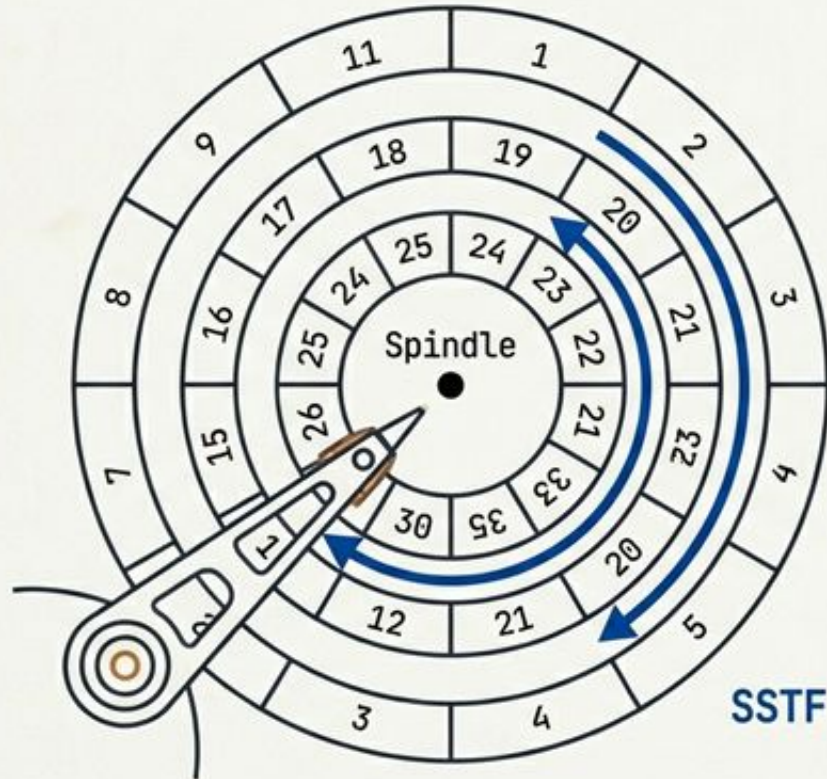
2. SSTF Scheduling

- It stands for shortest-seek-time-first (SSTF) algorithm.
- The SSTF algorithm selects the request with the minimum seek time from the current head position.
- Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position

98, 183, 37, 122, 14, 124, 65, 67 Consider now the Head pointer is in cylinder 53.



SSTF: Shortest Seek Time First



SSTF Path

Rule: Go to the nearest track.

WARNING

The Flaw: STARVATION.
If requests keep arriving at the inner track, the outer track is never served.

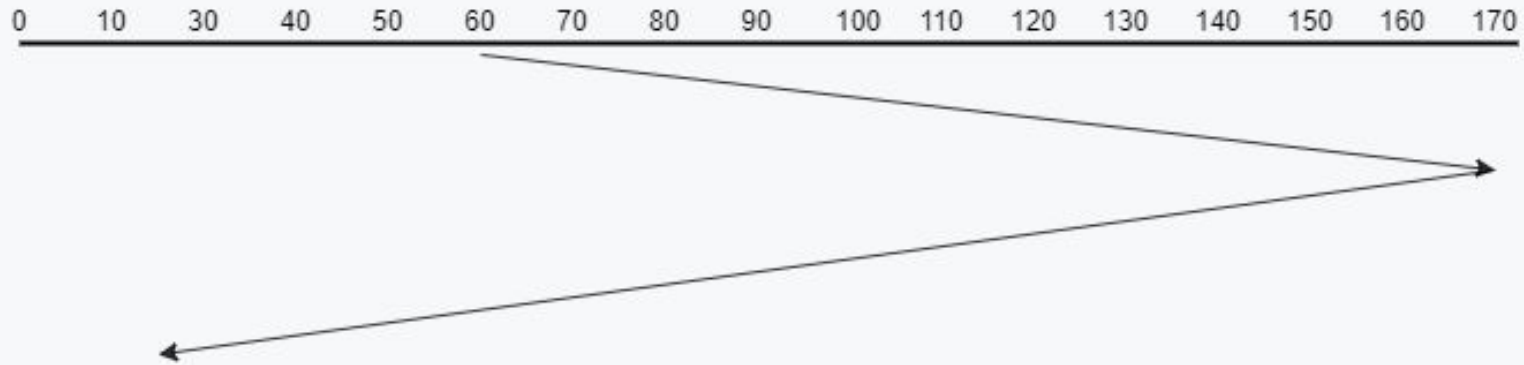
Blueprint Legend

DRAWN BY	DATE	REVISION
PROJECT		

3. SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- Illustration shows total head movement of 208 cylinders
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest

Eg. Suppose the order of requests are 70, 140, 50, 125, 30, 25, 160 and the initial position of the Read-Write head is 60. And it is given that the disk arm should move towards the larger value.

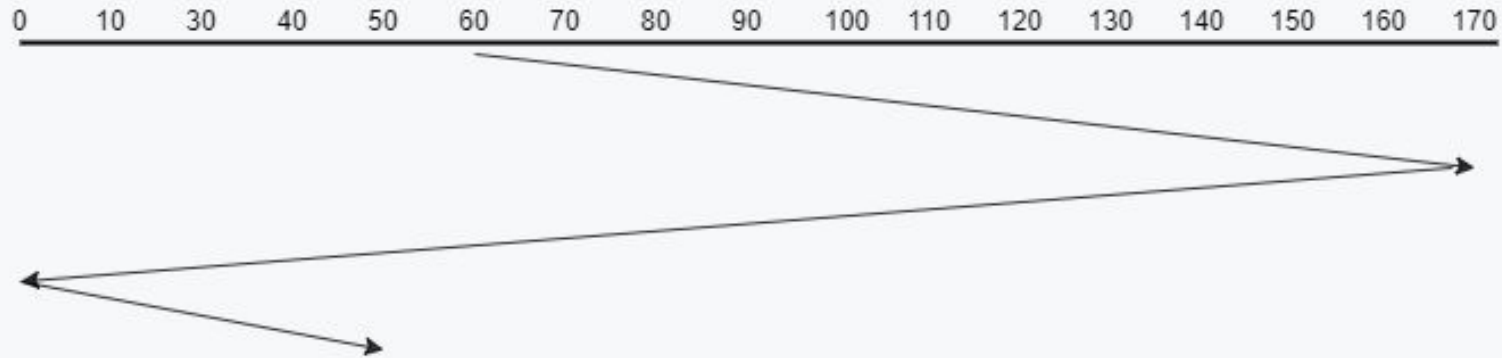


$$\text{Seek Time} = \text{Distance Moved by the disk arm} = (170-60) + (170-25) = 255$$

4. C-SCAN

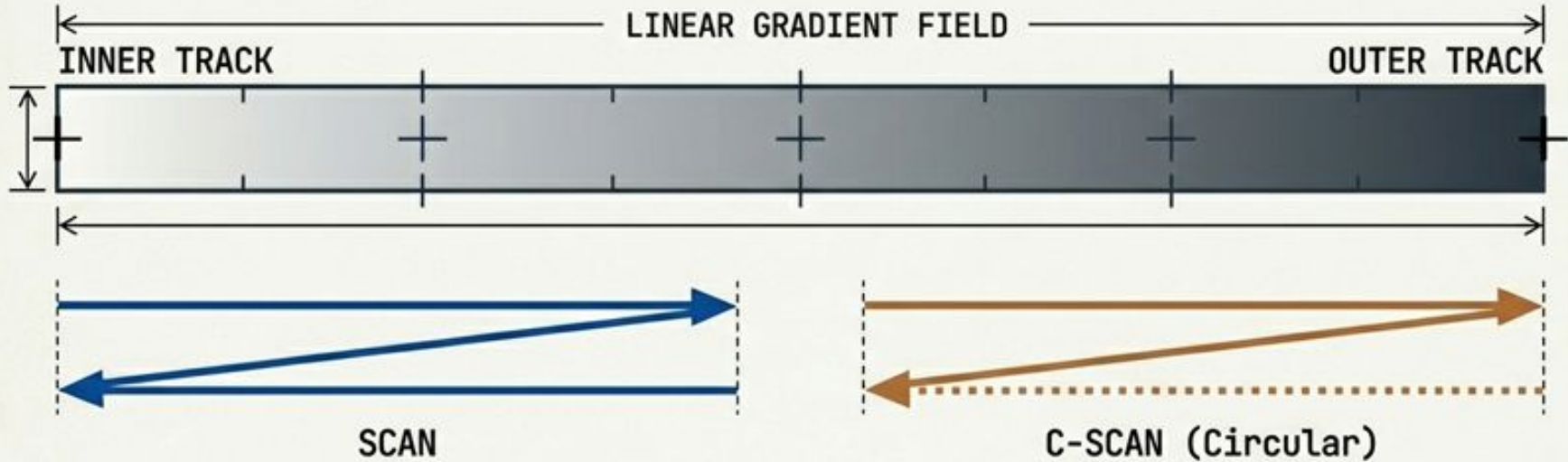
- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one

Eg. Suppose the order of requests are 70, 140, 50, 125, 30, 25, 160 and the initial position of the Read-Write head is 60. And it is given that the disk arm should move towards the larger value.



Seek Time = Distance Moved by the disk arm = $(170-60)+(170-0)+(50-0)=330$

The Elevator (SCAN & C-SCAN)



SCAN

C-SCAN (Circular)



- ⊕ SCAN: Sweeps back and forth. Services requests in track order.
- ⊕ C-SCAN (Circular): Sweeps one way only. Provides more uniform wait times (Fairness).
- ⊕ Advantage: Solves Starvation.
- ⊕ Disadvantage: Ignores Rotation.

Blueprint Legend

DRAWN BY	DATE	REVISION
PROJECT	 	

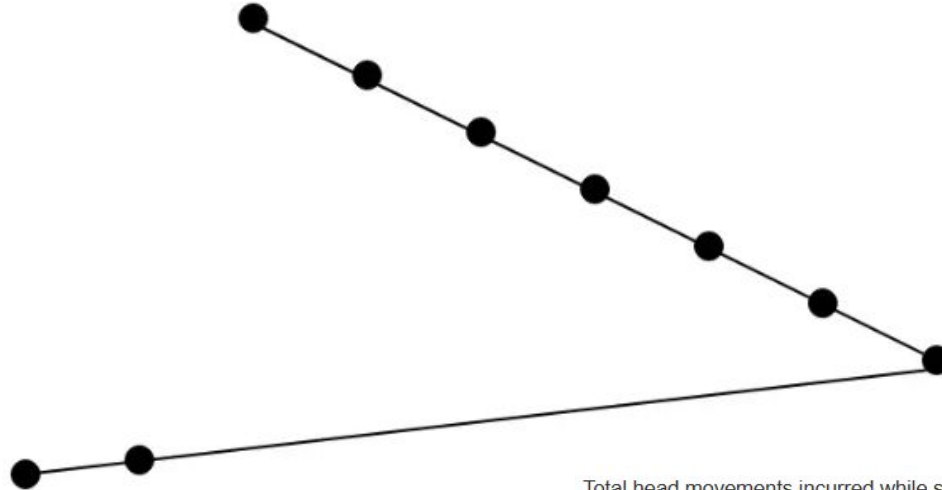
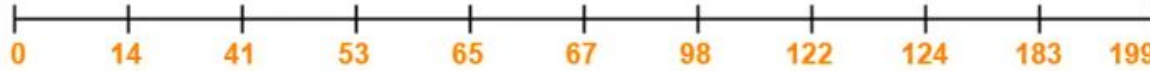
5. LOOK

- LOOK Algorithm is an improved version of the SCAN Algorithm.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end servicing all the requests in between.
- The same process repeats.

The main difference between SCAN Algorithm and LOOK Algorithm is-

- SCAN Algorithm scans all the cylinders of the disk starting from one end to the other end even if there are no requests at the ends.
- LOOK Algorithm scans all the cylinders of the disk starting from the first request at one end to the last request at the other end.

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.



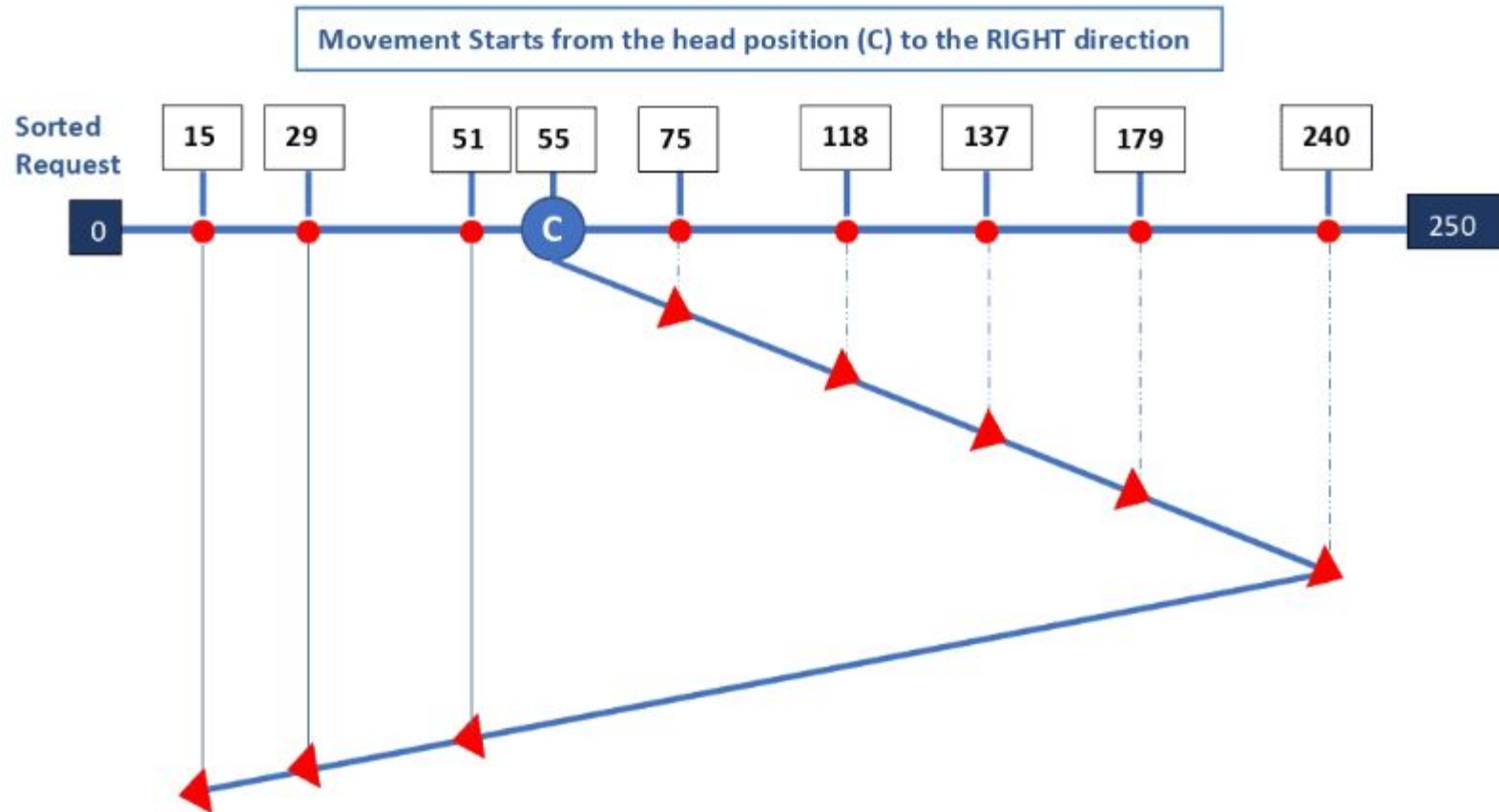
Total head movements incurred while servicing these requests

$$= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 41) + (41 - 14)$$

$$= 12 + 2 + 31 + 24 + 2 + 59 + 142 + 27$$

$$= 299$$

Let's consider an example of a disk with 251 tracks and a queue of disk access requests in the following order: 240, 94, 179, 51, 118, 15, 137, 29 75. The current position of the Read/Write head (C) is at track 55 and moving in the Right direction. The LOOK algorithm can be illustrated as shown in the below diagram:



The total head movement () can be calculated as follows:

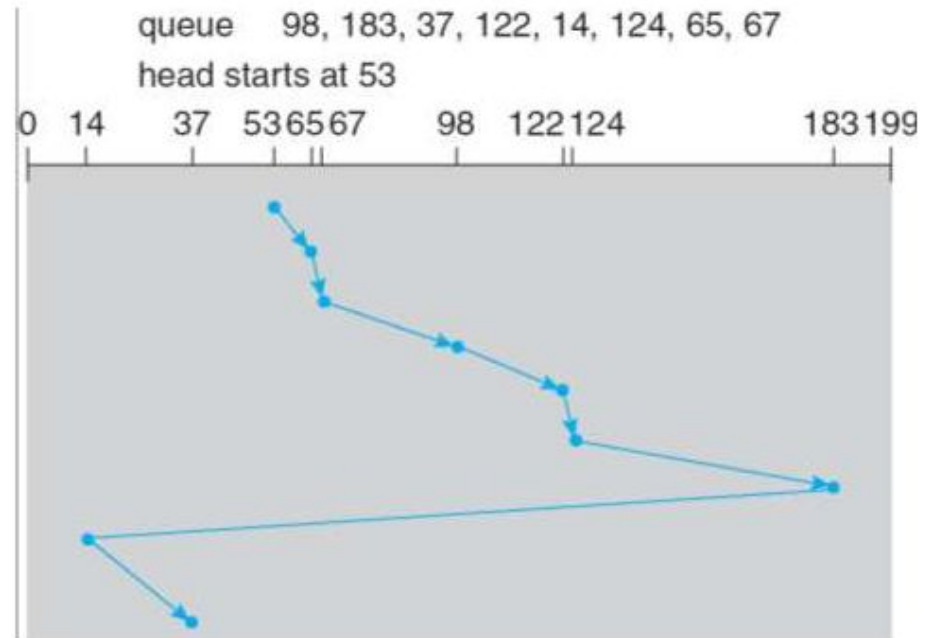
$$THM = (240 - 51) + (240 - 15)$$

$$THM = 189 + 225$$

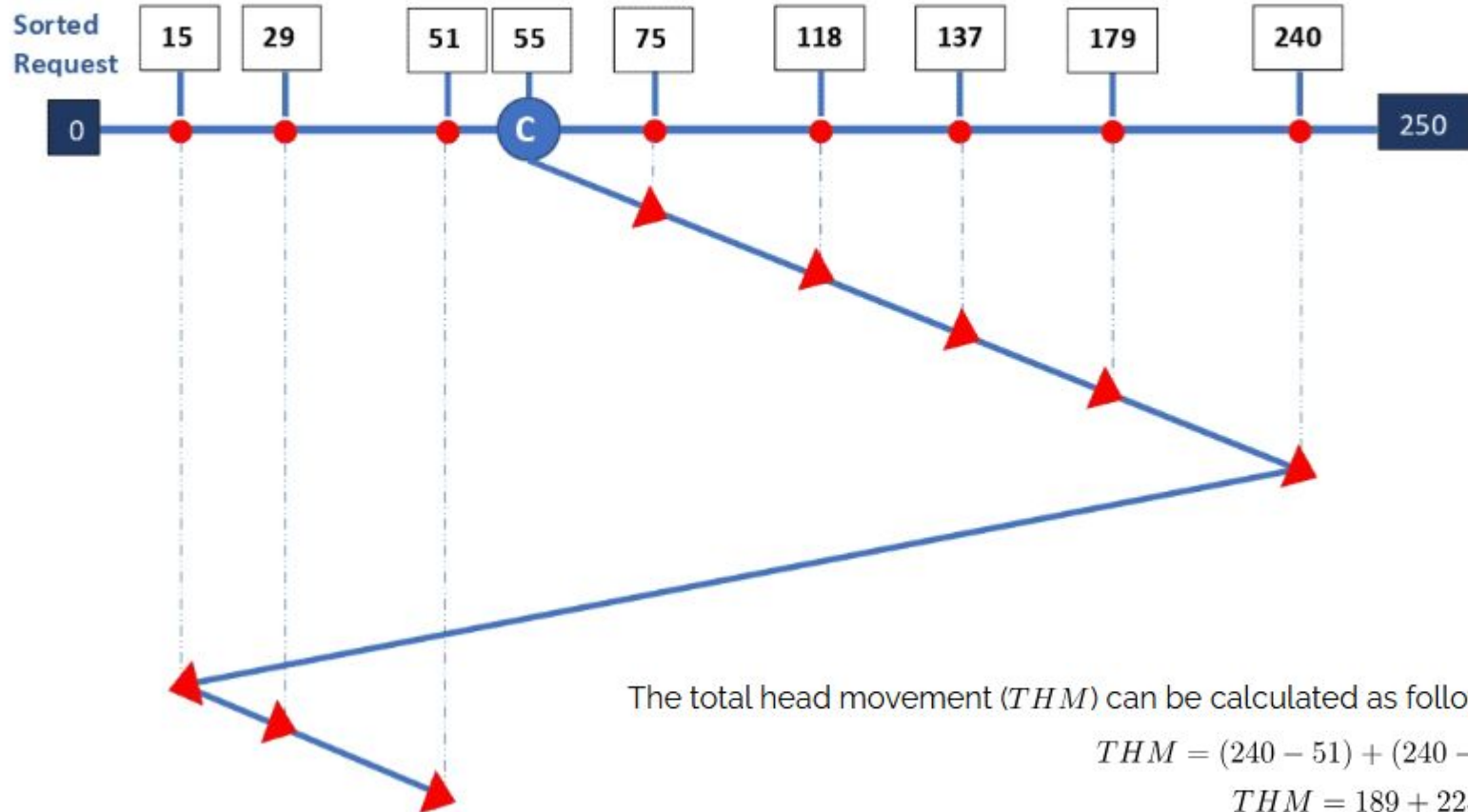
$$TotalHeadMovement = 414$$

5. C LOOK

- Version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.



Movement Starts from the head position (C) to the RIGHT direction



The total head movement (THM) can be calculated as follows:

$$THM = (240 - 51) + (240 - 15) + (51 - 15)$$

$$THM = 189 + 225 + 36$$

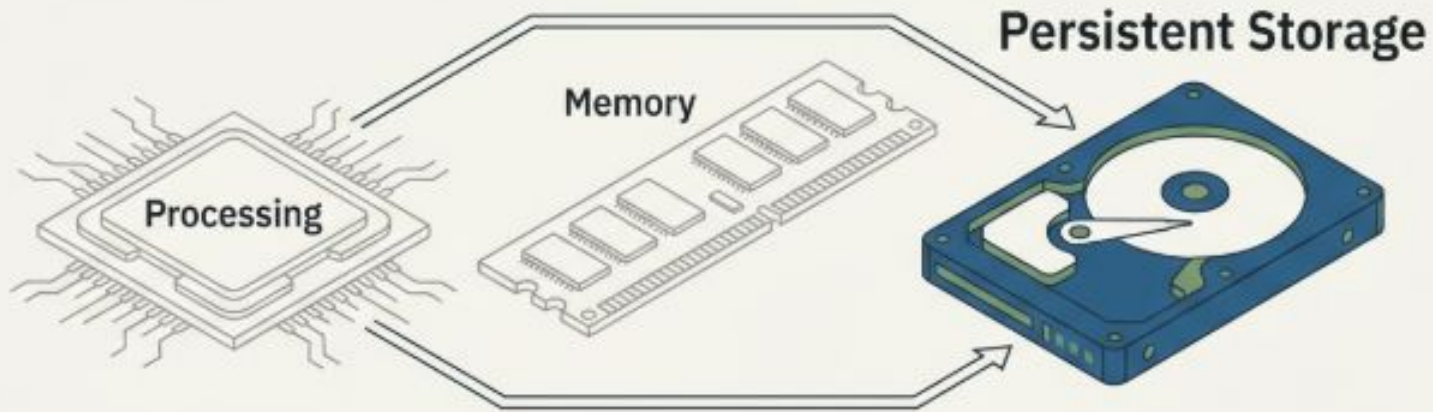
$$TotalHeadMovement = 450$$

Selecting a Disk-Scheduling Algorithm

- ➡ SSTF is common and has a natural appeal.
- ➡ SCAN and C-SCAN perform better for systems that place a heavy load on the disk.
- ➡ Performance depends on the number and types of requests.
- ➡ Requests for disk service can be influenced by the file allocation method.
- ➡ The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary.
- ➡ Either SSTF or LOOK is a reasonable choice for the default algorithm.

Files & Directories

The Final Virtualization Puzzle: Persistent Storage



Unlike memory, whose contents are lost when there is a power loss, a persistent-storage device keeps data intact. The OS must take extra care: this is where users keep data they really care about.

Virtualization of Storage

Bridging the gap between raw hardware and user data



The Reality

01001011	11001101	A3 B7 FF	00 1A C2	10101011	D4 E5 01
00 01 01	11010100	D4 E5 01	00 01 01	01110111	P6 E5 01
00 01 01	11010100	FF FE CD	AB 01 00	10101011	AB 01 00
11010100	FF FE CD	FE CD AB	00011010	10110011	B10 B011
00011010	AF FE CD	00011010	10110011	10110011	HF B1 1E2
10110011	AF 21 7E	99 11 21	FF FE CD	04111120	OE 32 58
11001011	ET 55 FF	FF FE CD	01001011	10001101	A3 B7 FF
01001011	11001101	A3 B7 FF	00 1A C2	10101011	D4 E5 01
10101011	D4 E5 01	00 01 01	11010100	01110111	P6 E3 00
10011011	AB B1 CD	00 01 01	FF FE CD	00011010	AB FE CC
11001101				1F FE 99	
01001011				2 D08 BEF	
11001111				1 28 18 91	
01001001				9 A3 B7 FF	
11001011				1 1010	
11001011				0 0000	
11001011				C FF FE CD	
01001101				1 4F 21 97	
01001101				1 A3 B2 FF	
01001101					
01001011	11001101	A3 B7 FF	00 1A C2	10101011	D4 E5 01
01001011	11001101	A3 B7 FF	00 1A C2	10101011	D4 E5 01
00 01 01	11010101	D4 E5 01	00 01 01	10101011	B 05 01 01
11010100	FF FE CD	FE CD AB	00011010	10110011	4F 21 7E 99
11010100	FF FE CD	FE CD AB	00011010	10110011	4F 21 7E 99
01001011	11001101	A3 B7 FF	00 1A C2	10011001	D4 E5 01
01001011	11001101	A3 B7 FF	00 1A C2	10101011	D4 E5 01
00 01 01	FF FE CD	D4 E5 01	00 01 01	10110011	B 05 01 01
11010100	FF FE CD	FE CD AB	00011010	10110011	4F 21 7E 99

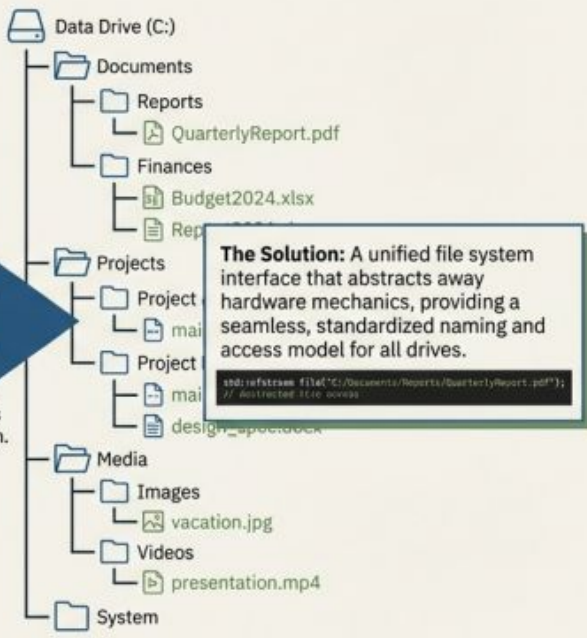
The Challenge: How do we manage a persistent device? What APIs can bridge the gap between raw hardware sectors and human-readable organization?

```
open("/dev/sda1", O_RDWR | O_DIRECT); // Direct sector access
```

OS Abstraction

Bridging raw hardware mechanisms to human-readable data organization.

The Illusion

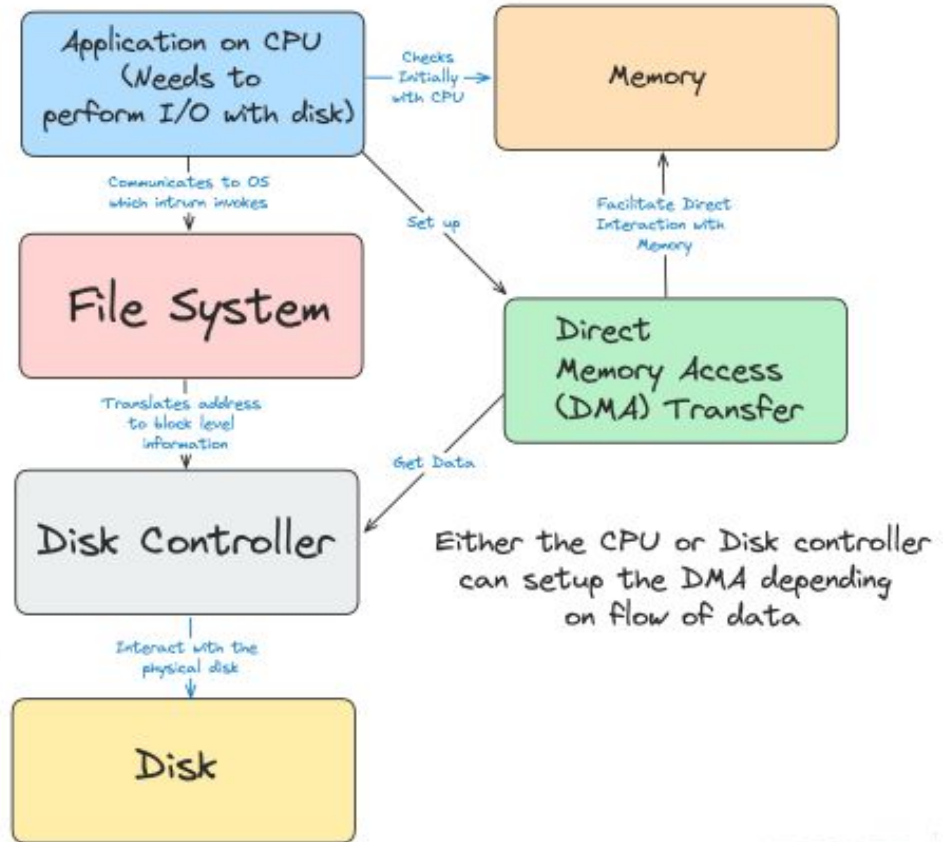


The Solution: A unified file system interface that abstracts away hardware mechanics, providing a seamless, standardized naming and access model for all drives.

```
std::ifstream f14("C:/Documents/Reports/QuarterlyReport.pdf");
// Abstracted file access
```

The flow of access

- Application performs read or write to a file
- CPU communicates to OS which invokes the File System (FS)
- The OS may check in its cache if its already there
- FS prepares block level information to disk controller
- A Direct Memory Access (DMA) is set up
- Disk controller performs the physical read or write based on commands from DMA and file system
- If its read, Disk -> DMA, for writes, DMA -> Disk



ISTEART

Files

- Linear array of bytes each of which can be read or written
- Each file has a human-readable name - “**sample.pdf**”
- Each file has a unique low-level name (not user given, OS given) - **inode number (i-number)**
- Type of the file is not the concern of the OS (image, code, etc)
 - File system should ensure that data is stored persistently
 - Also ensures that data is retrieved when requested
- Applications can worry about extensions and reading file in the way needed

Directories

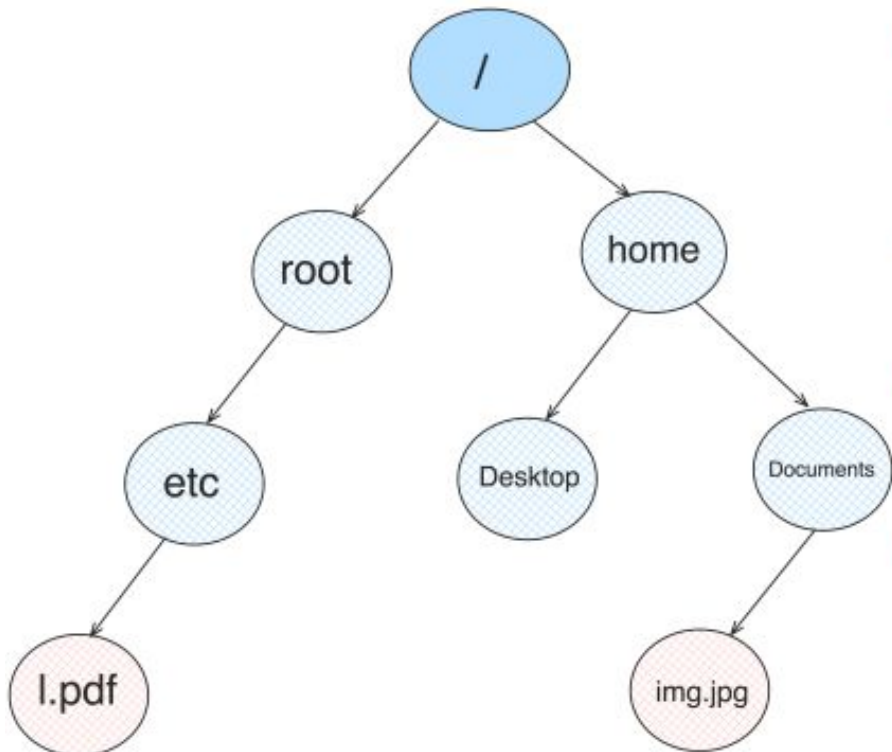
- A directory is just like a file
- It also has a low-level name: inode number
- Contains a list of pairs (**user readable file name, i-node number**)
 - Eg: consider a directory name **OSN**
 - (Lectures, 123) -> Directory
 - (OSN_L23.pdf, 326) -> File
- Basically directory is a special type of files with contents: files, directories and corresponding i-node numbers



The file abstraction

- File – linear array of bytes, stored persistently
 - Identified with file name (human readable) and a OS-level identifier (“inode number”)
 - Inode number unique within a file system
- Directory contains other subdirectories and files, along with their inode numbers
 - Stored like a file, whose contents are filename-to-inode mappings

The Unix Directory Tree



- Files and directories arranged in a tree
- Directory hierarchy starts at root directory - referred to as /
- Uses a separator to name subsequent directories
- **Absolute pathname** can be used:
 - /home/Documents/img.jpg
- File has two parts:
 - Arbitrary name - "img"
 - Type - ".jpg"
- Everything is an abstraction by OS

File System Interface

- Everything in Unix is virtually a file
- Mainly the file system has to provide three interfaces
 - **Creation of files** - Support creating files, allocate space
 - **Accessing files** - Reading and writing files
 - **Deletion of files** - Delete files and clear space
- Internally everything is 1s and 0s in the disk so File system has a big responsibility!



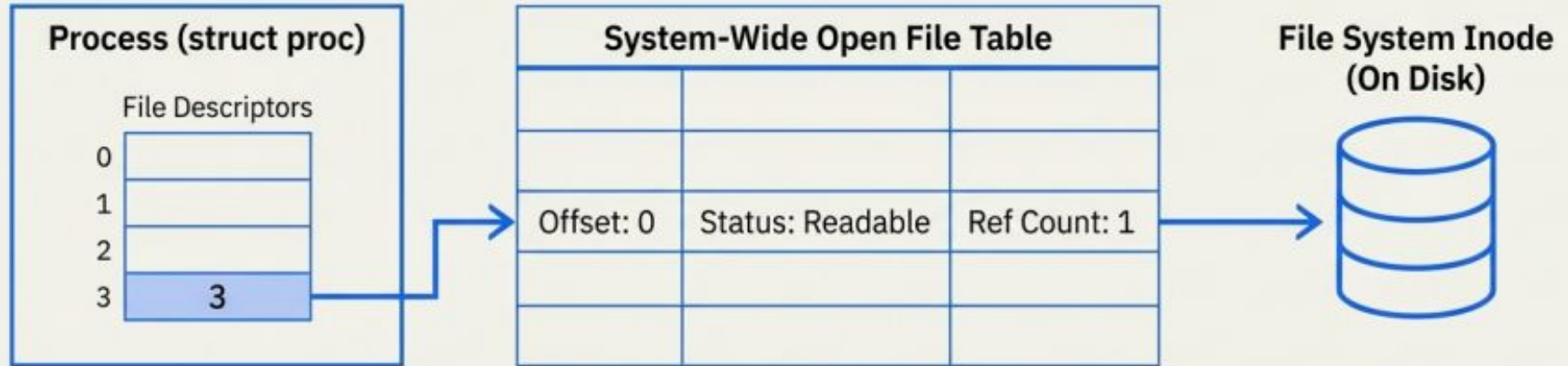
Creation Interface

- **open()** system call with flag to create file

```
int fd = open("sample", O_CREAT | O_WRONLY, O_TRUNC, S_IRUSR | S_IWUSR);
```

- O_CREAT: creates a file if it does not exist
- O_WRONLY: file is write only
- O_TRUNC: truncates file to zero bytes if it already exists
- S_IRUSR or S_IWUSR: permissions - make file readable or writeable
- The call returns a number, **file descriptor**: operations on file uses the file descriptor
- Existing files must be opened before they can be read or written
- **close()**: closes the file

File Descriptors act as capabilities pointing to the Open File Table



File Descriptor (FD):

An integer private to each process. It acts as an opaque handle (a capability) granting the power to perform I/O operations.

Open File Table:

A shared structure that tracks current offset, readability/writability, and reference counts for all open files across the entire system.

Random Reading and Writing

- In general file is accessed sequentially
 - Read/write from beginning to end
- What if it needs to be randomly accessed for read/write?
 - `lseek()` system call - seek to random offset
 - Start reading and writing from random offset
 - **`off_t lseek(int flides, off_t offset, int whence);`**

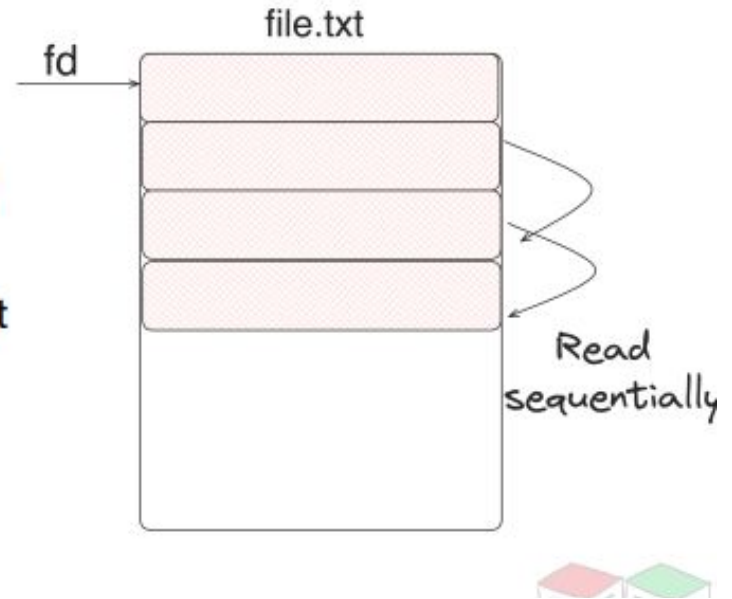
`lseek` has nothing to do with disk seek!

- flides - file descriptor
- off_t - moves pointer to a given offset,
- Whence - determines how seek is performed (from an offset, from given + some offset or size of file + offset)



Reading and Writing Files

- `read ()` / `write ()` system calls: Reading/writing files
 - Three arguments: file descriptor, buffer with data, size
 - Buffer - where data will be placed and size - size of buffer
 - Reading and writing happens sequentially by default
 - Successive read/write calls fetches from the offset that is being used
- Every process has three files opened - stdin, stdout, stderr with fd 0, 1 and 2



A Simple Example - Normal Read

System calls	Return Code	Current Offset
<code>fd = open("file.txt", O_RDONLY);</code>	3	0
<code>read (fd, buffer, 100);</code>	100	100
<code>read (fd, buffer, 100);</code>	100	200
<code>read (fd, buffer, 100);</code>	100	300
<code>read (fd, buffer, 100);</code>	0	300
<code>close(fd);</code>	0	-

- Offset is initialised to 0 when opened
- For each read call, the offset is incremented fixed value - sequentially
- At the end, 0 denotes the read has been completed



A Simple Example - Seeking

System calls	Return Code	Current Offset
<code>fd = open("file.txt", O_RDONLY);</code>	3	0
<code>lseek(fd, 200, SEEK_SET);</code>	200	200
<code>read (fd, buffer, 50);</code>	50	250
<code>close(fd);</code>	0	-

- Offset is initialised to 0 when opened
- lseek sets the offset to 200
- Read call, reads the next 50 bytes and updates offset

There is a buffer - How to write immediately?

- Regular writes, `write()` puts the data to buffer => some point it will be written to persistent storage
- This is done for performance enhancement (keep in buffer for 5 to 30 seconds)
- Some applications require more real-time guarantees
- System call: `fsync(int fd)`: returns 0, once write is complete
- Sometimes `fsync` has to be called on directory itself that contains the file
- This ensures that file is on disk

```
fsync example

#include <stdio.h>

int main ()
{
    int fd = open("sample", ...);
    assert (fd > -1);
    int rc = write (fd, buffer, size);
    assert (rc == size);
    rc = fsync(fd);
    assert(rc == 0);
    return 0;
}
```



Metadata of files

- File system stores fair amount of data about files
- Information include: file size, last access, last modified, user id of the owner, links count, pointers to data blocks, etc.
- This metadata is stored by file systems in a structure called **inode**
- **Inode** - persistent data structure used by the file system
 - They store all the metadata information for a file
 - They are stored in the disks but copies are cached to main memory when needed!



Interface for Directories

- Directories can also be accessed like files
 - Operations like create, open, read, close
- Create directory - **mkdir()** system call, when created its empty. It has two entries
 - “.” And “..” Itself and the parent directory respectively
- Listing all the directories - **ls** command (internally - **opendir()**, **readdir()** and **closedir()**)
- What about **rm *** and **rm -rf *** ? - Powerful double-edged sword!
- Directory entry contains information such as name, l-node number,
- Deleting directory - **rmdir()** - System call and command have same name



Key Directory System Calls:

- `DIR *opendir(const char *name);` Opens a directory stream corresponding to the directory name provided and returns a pointer to the directory stream.
- `int closedir(DIR *dirp);` Closes the open directory stream associated with the directory pointer `dirp` to release associated resources
- `mkdir(const char *pathname, mode_t mode);` Creates a new, empty directory.
- `rmdir(const char *pathname);` Removes an empty directory.

The **rm** command in Linux is used to delete files and directories permanently from the file system. It removes data immediately without sending it to any recycle bin, so deleted files cannot be recovered.

-rf (recursive + force): Deletes directories and files recursively without any confirmation

Operations on directories

- Every file is identified by a unique **inode number** in filesystem
- Directory is a special file that contains mappings between filenames and inode number of the file
- Directories can also be accessed like files, e.g., create, open, read, close
- For example, the “ls” program opens and reads all directory entries
 - Directory entry contains file name, inode number, type of file (file/directory) etc.

```
int main(int argc, char *argv[]) {  
    DIR *dp = opendir(".");  
    assert(dp != NULL);  
    struct dirent *d;  
    while ((d = readdir(dp)) != NULL) {  
        printf("%lu %s\n", (unsigned long) d->d_ino, d->d_name);  
    }  
    closedir(dp);  
    return 0;  
}
```

Image credit: OSTEP

Hard Links

- Hard linking creates another file that points to the **same i-node number** (hence same underlying data)
- Assume a file, “file1” which just contains a string “test” - What if we need file2 linked to this?
- Another file that links to this can be created using **link()** call - **ln** command
- Essentially both files have same underlying data - just two different user-given names
- I-node maintains a **link count**, file deleted only when no further links to it
- One can only unlink file, OS decides when to delete

```
Hard links

prompt> echo hello > file1
prompt> cat file1
hello OSN Students
prompt> ln file1 file2
prompt> cat file2
hello OSN Students
```

```
Hard links

prompt> rm file1
removed "file1"
prompt> cat file2
hello OSN Students
```

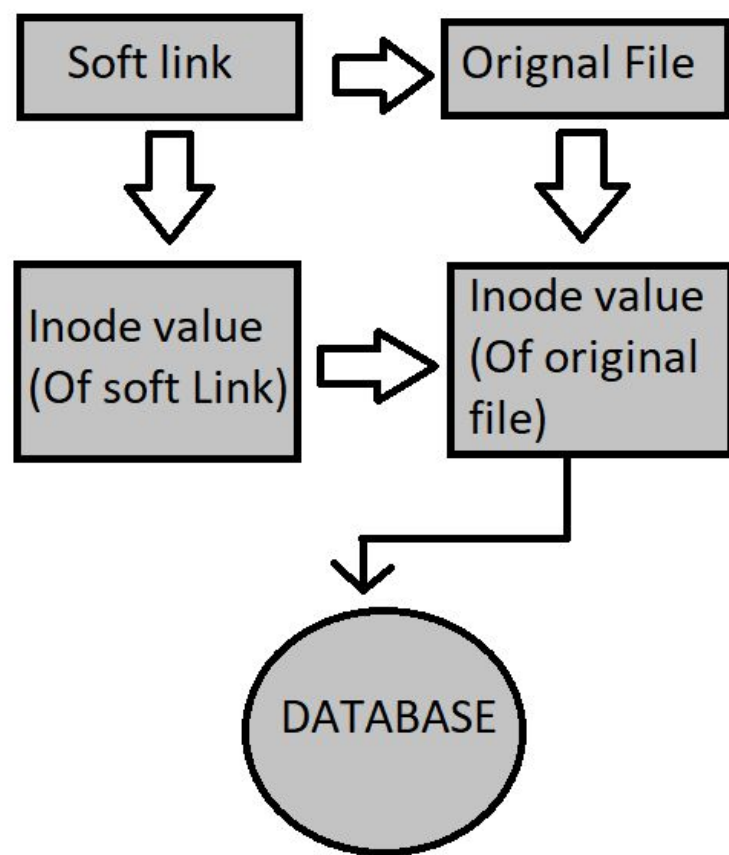
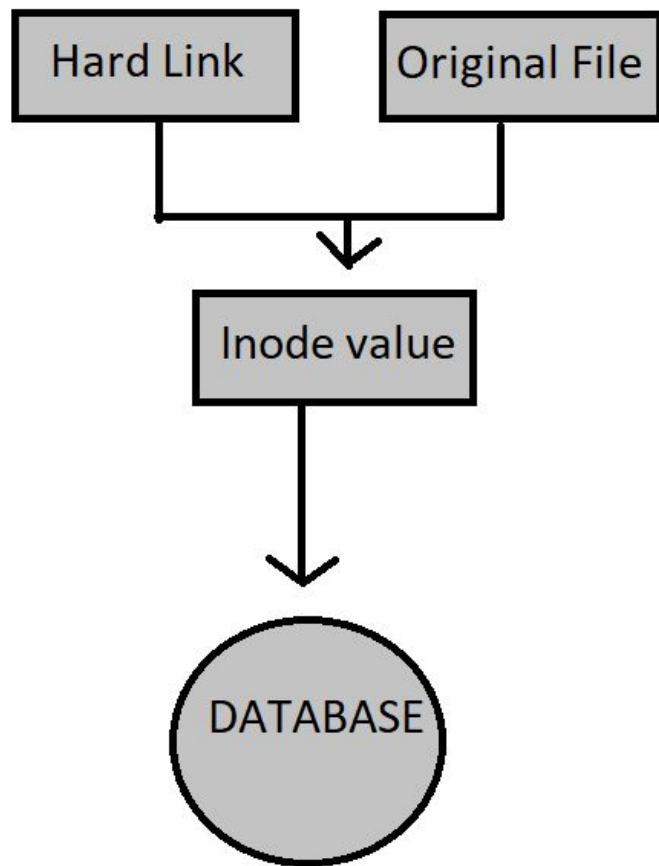
Symbolic Links or Soft Links

- Another way to create link - This time in much simpler
 - Hard links are limited - link to directory not possible
 - Hard link to files in other disk not possible
 - I-node is unique within a file system
- **Symbolic link or soft link** creates a file by itself
 - The name can be different
 - **i-node number will be different**
 - If the main file is deleted, link points to an invalid entry: **dangling reference**

```
Soft Links

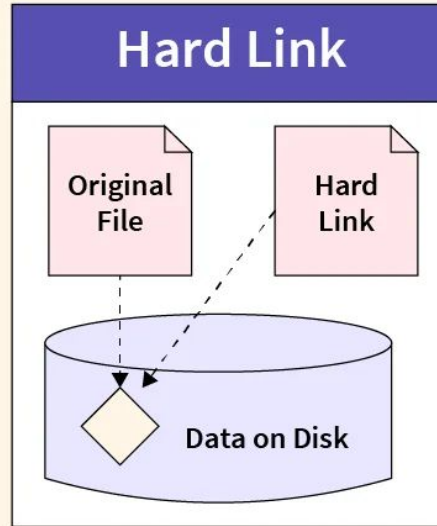
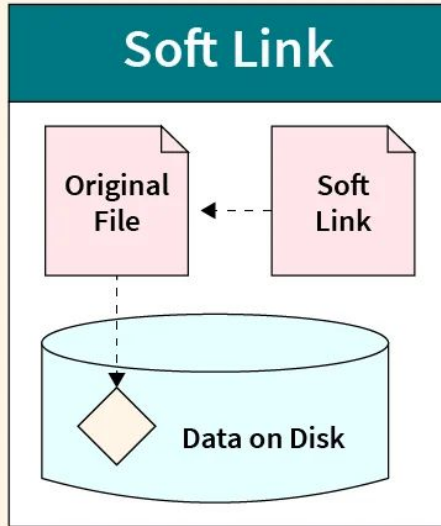
prompt> echo "Hello OSN" > file1
prompt> cat file1
Hello OSN
prompt> ln -s file1 file2
prompt> cat file2
Hello OSN
prompt> rm file1
prompt> cat file2
cat: file2: No such file or directory
```





Difference : hard link and symbolic link

Hard link	Symbolic link
Does not create a new inode	Create a new inode
Cannot link directories unless it is done by root	Can link directories
Cannot link files across file systems	Can link files across file system
Increase hard link count of the linked inode	Does not change hard link count of the linked inode



Mounting a file system

- Mounting a file system connects the files to a specific point in the directory tree

```
prompt> mount -t ext3 /dev/sda1 /home/users
prompt> ls /home/users/
a b
```

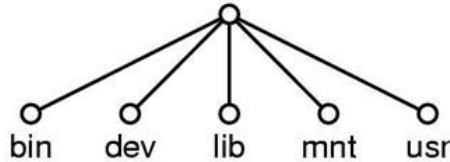
- Several devices and file systems are mounted on a typical machine, accessed with `mount` command

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

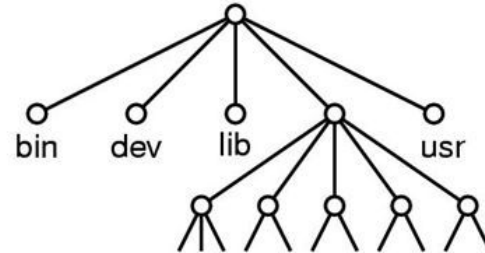
File System Mounting

- Mount allows two FSES to be merged into one
 - For example you insert your floppy into the root FS

`mount("/dev/fd0", "/mnt", 0)`



(a)



(b)

Beyond Files and Directories

- Mounting a file system connects the files to specific point in the directory tree

```
mount -t ext3 /dev/sda1 /home/users
```

```
ls /home/users
```

- Assembling directory tree from underlying file system
 - Accomplished by mounting the file system
 - Two tasks: **making the file system** and **mounting**
- Several devices and file systems are mounted on a typical machine
- Can be accessed with mount command



Memory mapping a file

- Alternate way of accessing a file, instead of using file descriptors and read/write syscalls
- `mmap()` allocates a page in the virtual address space of a process
 - “Anonymous” page: used to store program data
 - File-backed page: contains data of file (filename provided as arg to `mmap`)
- When file is `mmap`ed, file data copied into one or more pages in memory, can be accessed like any other memory location in program

How can we build a simple File System?

What structures are needed in disk and how to access?

File System

File System

- Organization of files and directories on disk
- OS has one more file systems
- File system is **pure software**, features:
 - Provide support for the sys calls
 - Manage the storage of data
 - No additional hardware support
- Great deal of **flexibility** when building FS
- Details vary with various file systems



File System

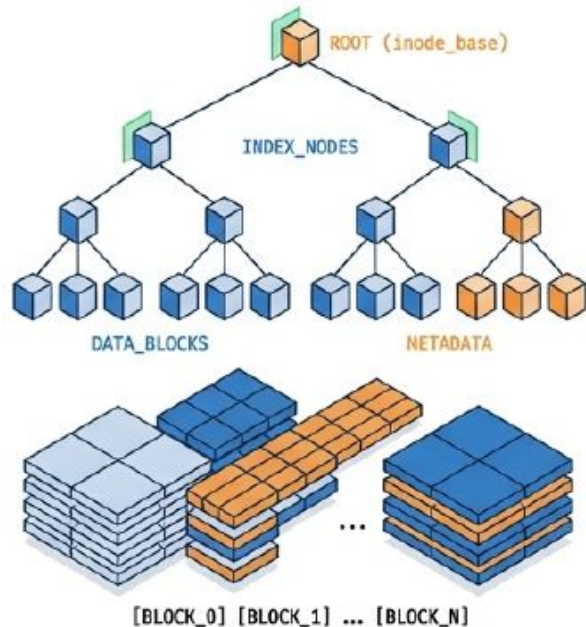
- An organization of files and directories on disk
- An OS can implement one or more file systems
- Disks expose a set of blocks (usually 512 bytes)
- File system organizes files onto blocks
 - System calls translated into reads and writes on blocks
- We will study the following concepts about file systems
 - The file abstraction and system call API exposed to users
 - Data structures to organize data and metadata on disk and memory
 - Implementation of system calls like open, read, write using the data structures

The Two Pillars of File Systems

To understand file systems, you must build a mental model around two distinct aspects:

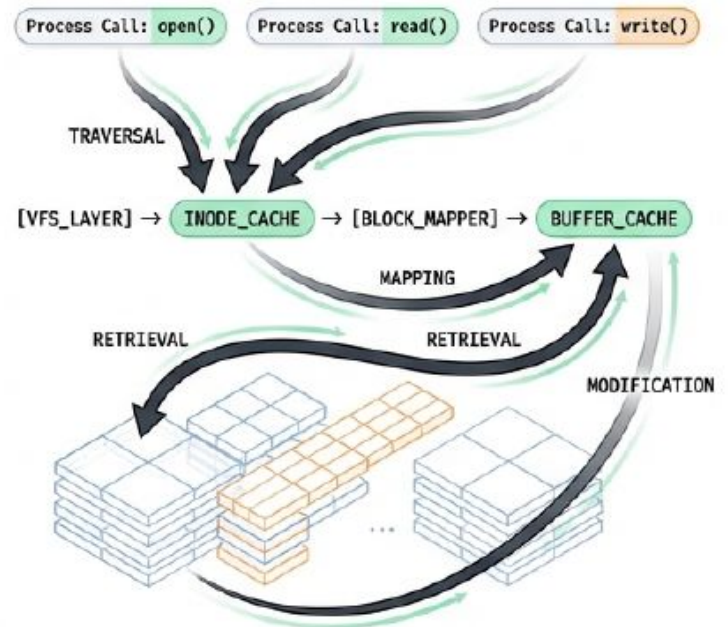
Pillar 1: Data Structures (The Anatomy)

What on-disk structures organize the data and metadata? (Arrays, blocks, trees).



Pillar 2: Access Methods (The Physiology)

How do process calls (open, read, write) map to these static structures in motion?



Breaking down into two main aspects

- Lets try building a simple file system - **Very Simple File System (VSFS)**
- In any FS, two key things make the difference

Data Structures

- What types of on-disk data structures are utilized by the file system to organise its data and metadata?
- VSFS can make use of simple structures like array of blocks (complex ones: trees)

Access Methods

- How can the calls like `open()`, `read()`, `write()`, etc made by process be mapped?
- Which structures are read during the execution of a system call?
- What about the efficiency?



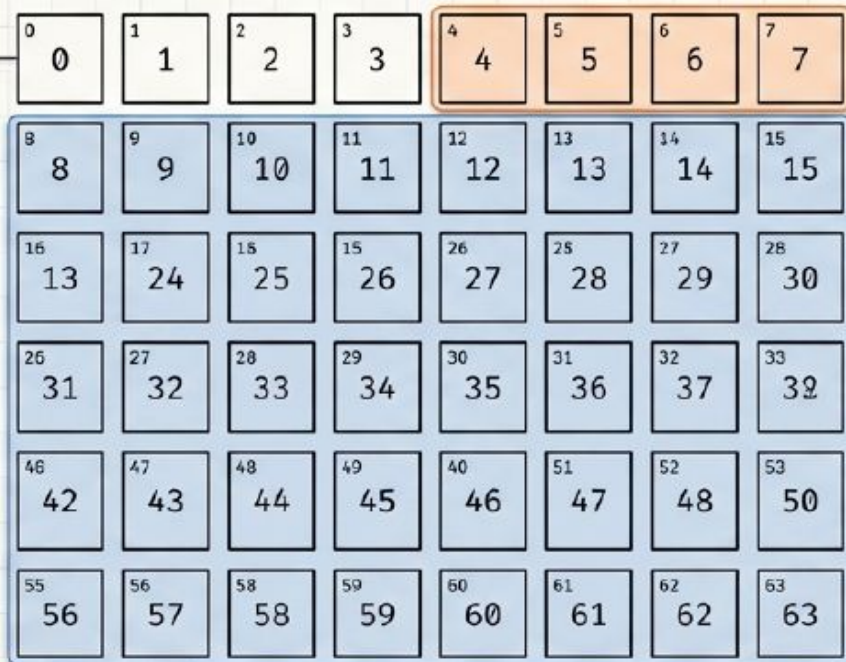
File System Data Structures: Index node (inode)

- Files are variable sized, split into **fixed size blocks**, stored non-contiguously
 - Much like how memory image of process is split into fixed size pages
 - Fixed size blocks avoids external fragmentation of disk storage
- For every file, **index node (inode)** keeps track of all the block numbers (locations on disk) where the file data is stored
 - Equivalent to a page table which keeps track of physical frame numbers
- Inode of a file is also stored in disk blocks
 - Much like how page table is stored in one or more pages hierarchically
- Inode stores all **metadata** about file (size, permissions, time of last access/modification, disk block numbers of file data, ..)

Macro Anatomy: Dividing the Raw Disk

The Block

Simple file systems use a single block size. We will use a standard 4 KB.



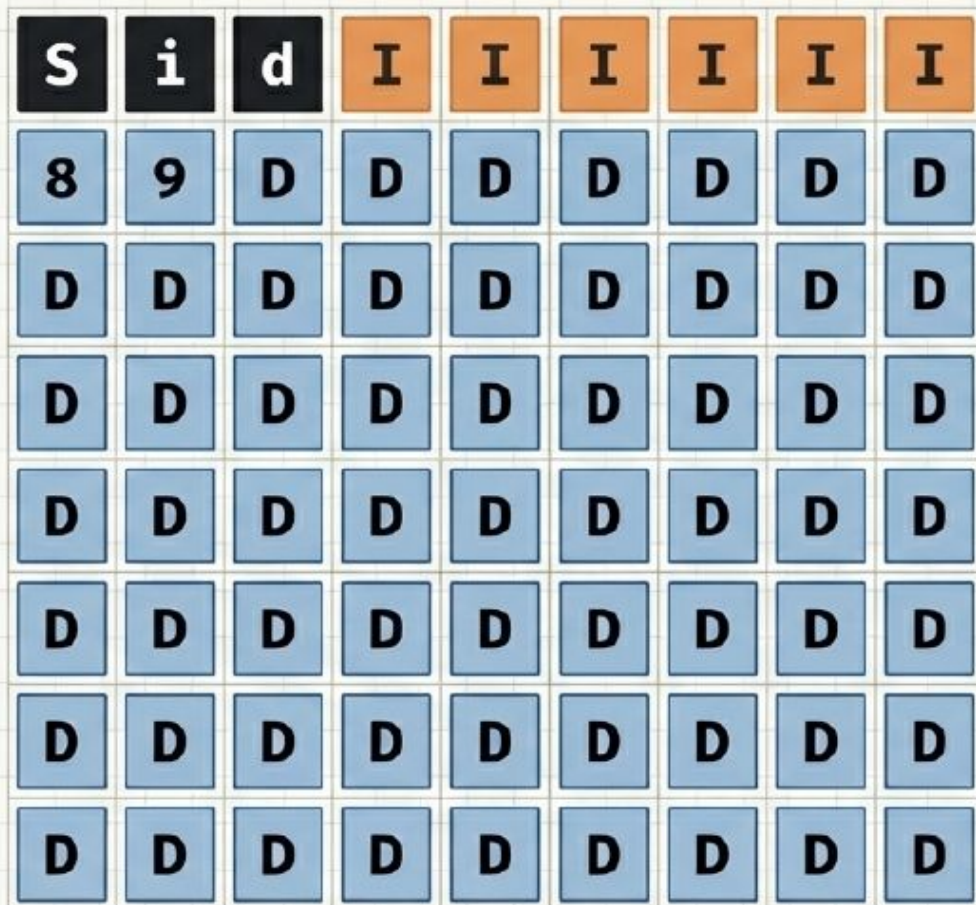
The Inode Table

A small reserved space for metadata tracking each file.

The Data Region

Most space is dedicated to user data.

The Complete On-Disk Layout



Key/Legend

S (Superblock):

Contains FS metadata (total blocks, inode table start address, magic number). Read first when mounting.

i/d (Bitmaps):

Allocation structures tracking free vs. in-use inodes and data blocks.

I (Inodes):

The metadata nodes for individual files.

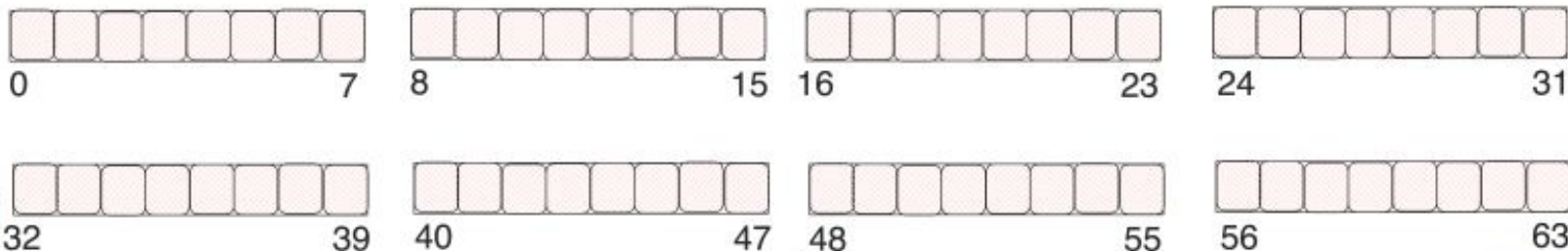
D (Data):

The actual user contents.

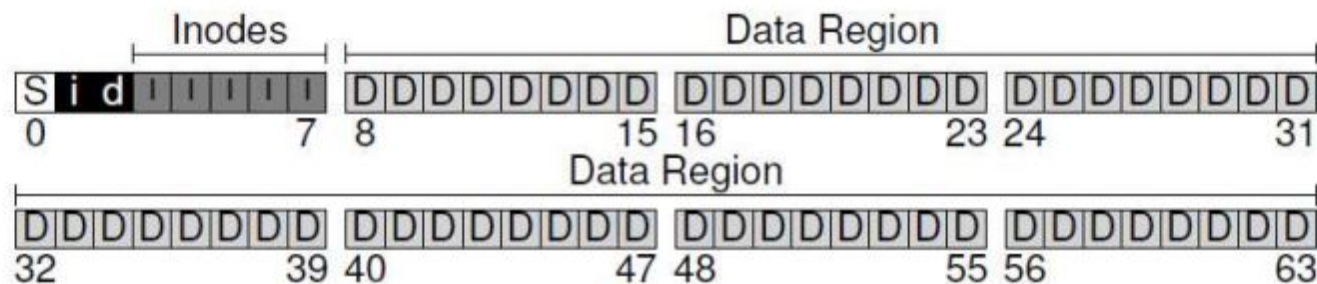
Data structures

On-disk organisation of VSFS

- Remember: Disk exposes a set of **blocks**
- File system has to organise the files into blocks - **Data**
- The information about the files also have to be stored - **metadata**
- Consider a disk with 64 blocks, each of size 4 KB (same sized blocks)
 - 0 to 63 in general **0 to N-1**
 - **What needs to be stored in these blocks?**



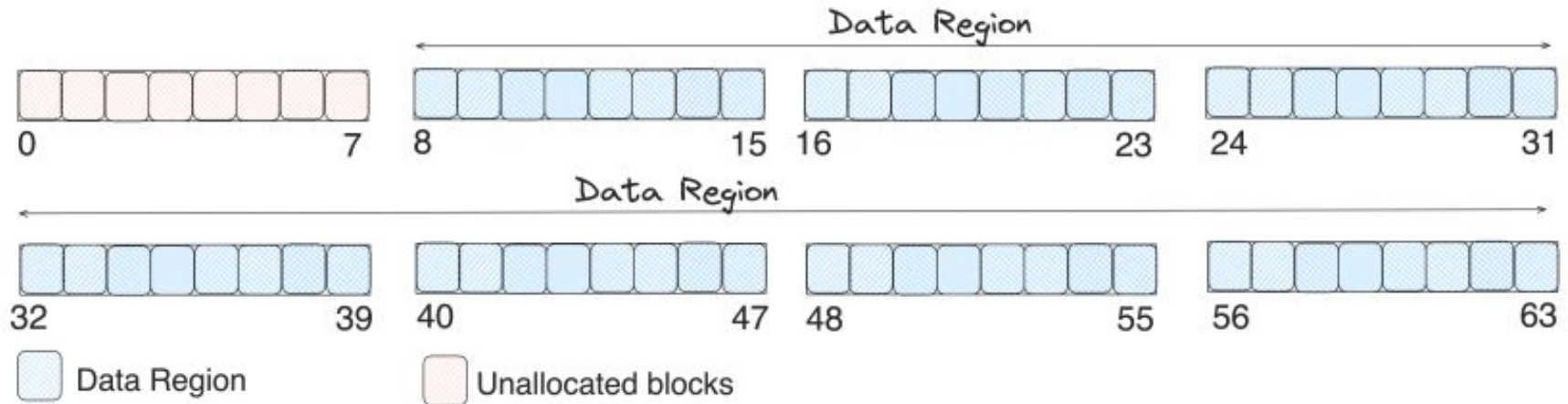
Example: a simple file system



- Data blocks: file data stored in one or more blocks
- Metadata about every file stored in inode
 - Location of data blocks of a file, permissions etc.
- Inode blocks: each block has one or more inodes
- Bitmaps (or free lists): indicate which inodes/data blocks are free
- Superblock: holds master plan of all other blocks (which are inodes, which are data blocks etc.)
- Structure imposed when hard disk is “formatted” with a file system

Data Region in the File System

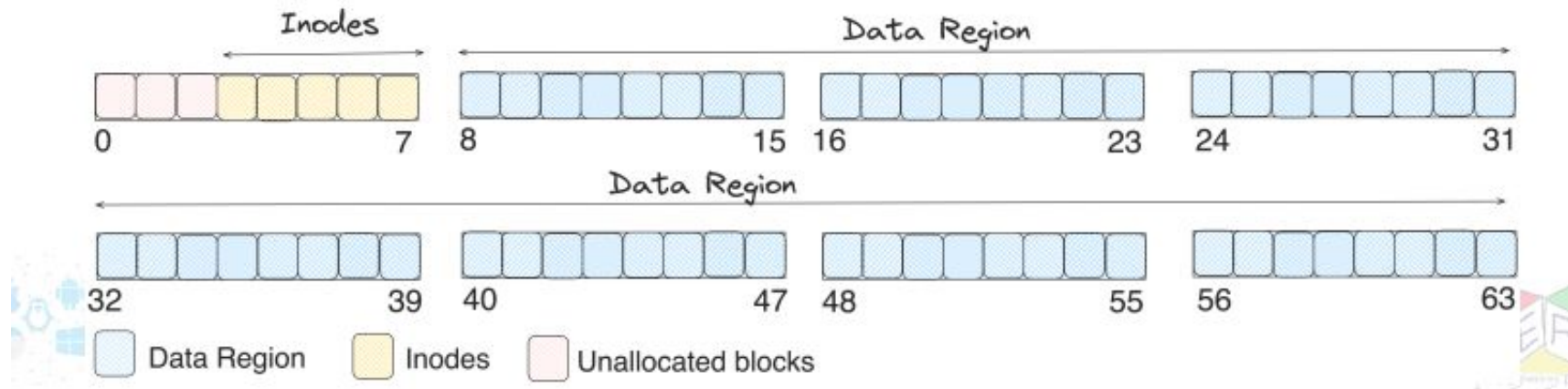
Some blocks need to be reserved for storing data - **data region**



- More information needs to be stored about where the data blocks are located, type of file, etc
- The inodes need to be stored

Some Space for Inodes!

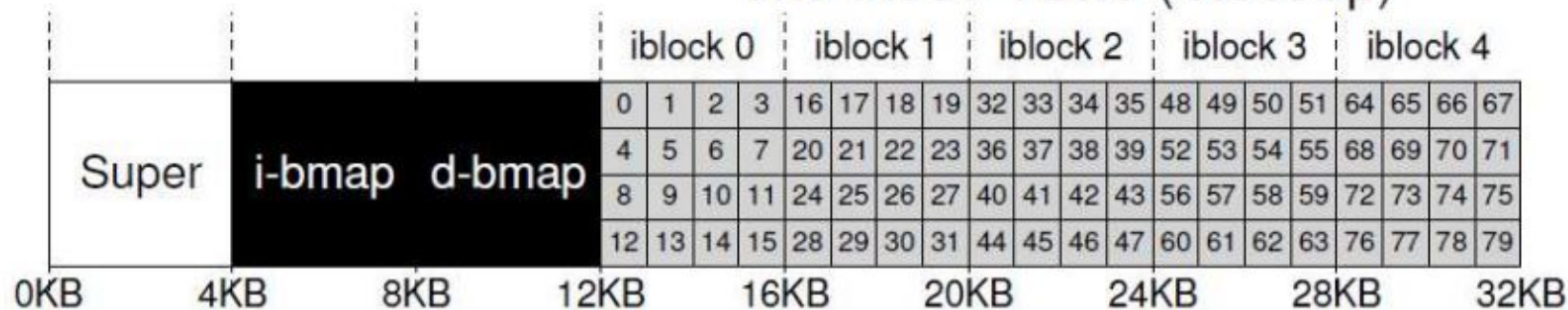
- Dedicate some space for inode table
 - This can hold an array of on-disk inodes
 - Consider each inode takes 256 bytes and 5 blocks are dedicated
 - Each block can hold 16 inodes => file system can hold 80 files



Inode on disk

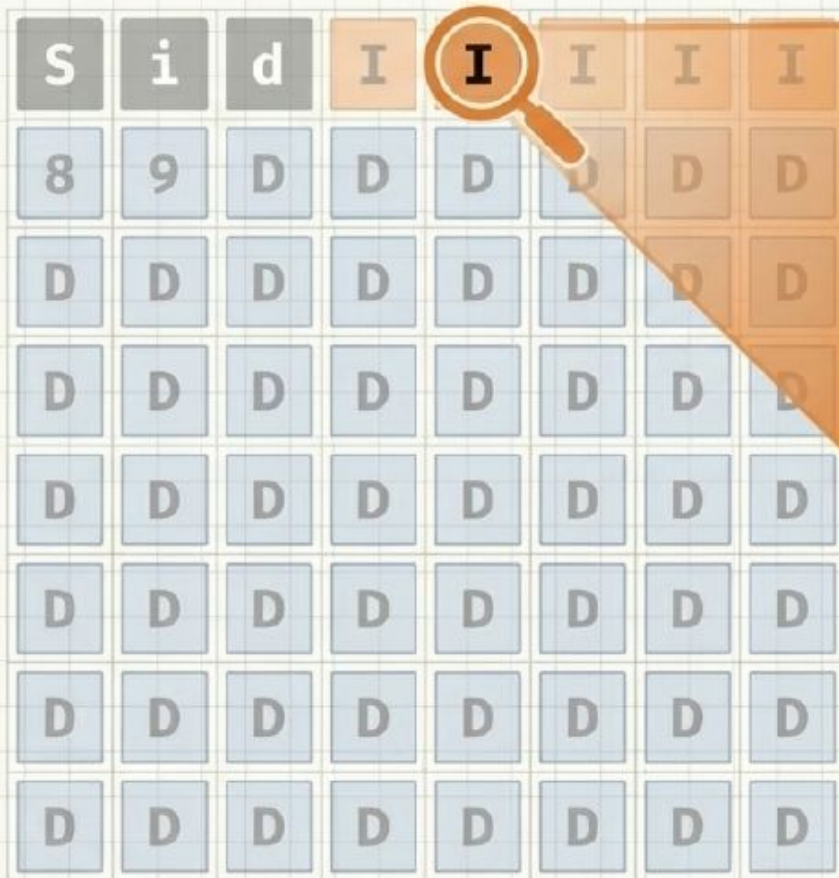
- Usually, inodes (index nodes) stored in array contiguously on disk
 - Inode number of a file is index into this array, uniquely identifies file

The Inode Table (Closeup)



- What does inode store?
 - File metadata: permissions, access time, etc.
 - Pointers (disk block numbers) of file data

Micro Anatomy: The Inode (Index Node)



X-Ray

INODE DETAILS

Type: — Regular File / Directory

Size: — Bytes in the file

UID/GID: — Ownership and permissions

Times: — Created, Modified, Accessed

Pointers: — Disk addresses for the data blocks

Inside each inode is virtually all the information you need about a file—except its human-readable name.

```
blk = (inumber * sizeof(inode)) / blockSize;
```


Structure of inode

- Inode stores block numbers hierarchically
 - Inode contains the block numbers of first few blocks of a file (**direct blocks**)
 - If direct blocks are full, inode contains block number of **single indirect block**, which contains block numbers of next few blocks of a file
 - If single indirect block is full, inode contains block number of **double indirect block**, which contains block numbers of more single indirect blocks
 - Triple indirect block can also be used for large files
- Not a symmetric hierarchical structure like page table
 - Most files are small, so first few block numbers of a file are made available easily without accessing multiple levels of inode
- Accessing a file from disk may require multiple disk accesses for inode

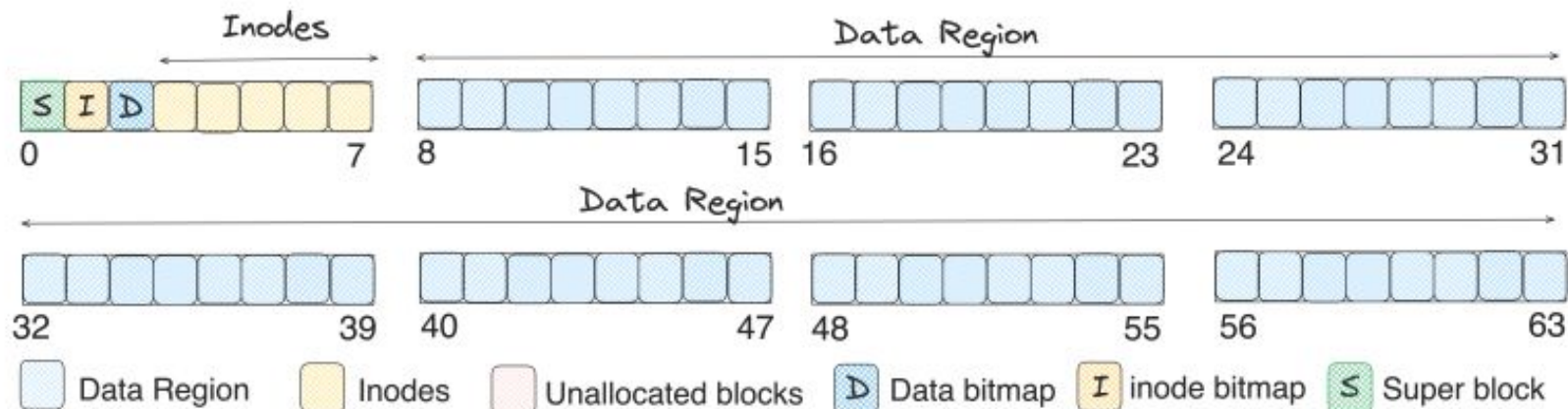
We still miss something!

- FS needs some mechanism to track which inodes are free and which data blocks are free
- How can such information be tracked? Which are free and which are available?
 - Use **bitmaps**, each bit can be used to denote if corresponding block is free or not
 - 0 if the corresponding block is free
 - 1 if the corresponding block is allocated
 - In our vsfs - 80 inodes and 56 blocks for data
 - Assume that we dedicate **two blocks for bitmaps** for **inode** and **data**

28



A more complete representation

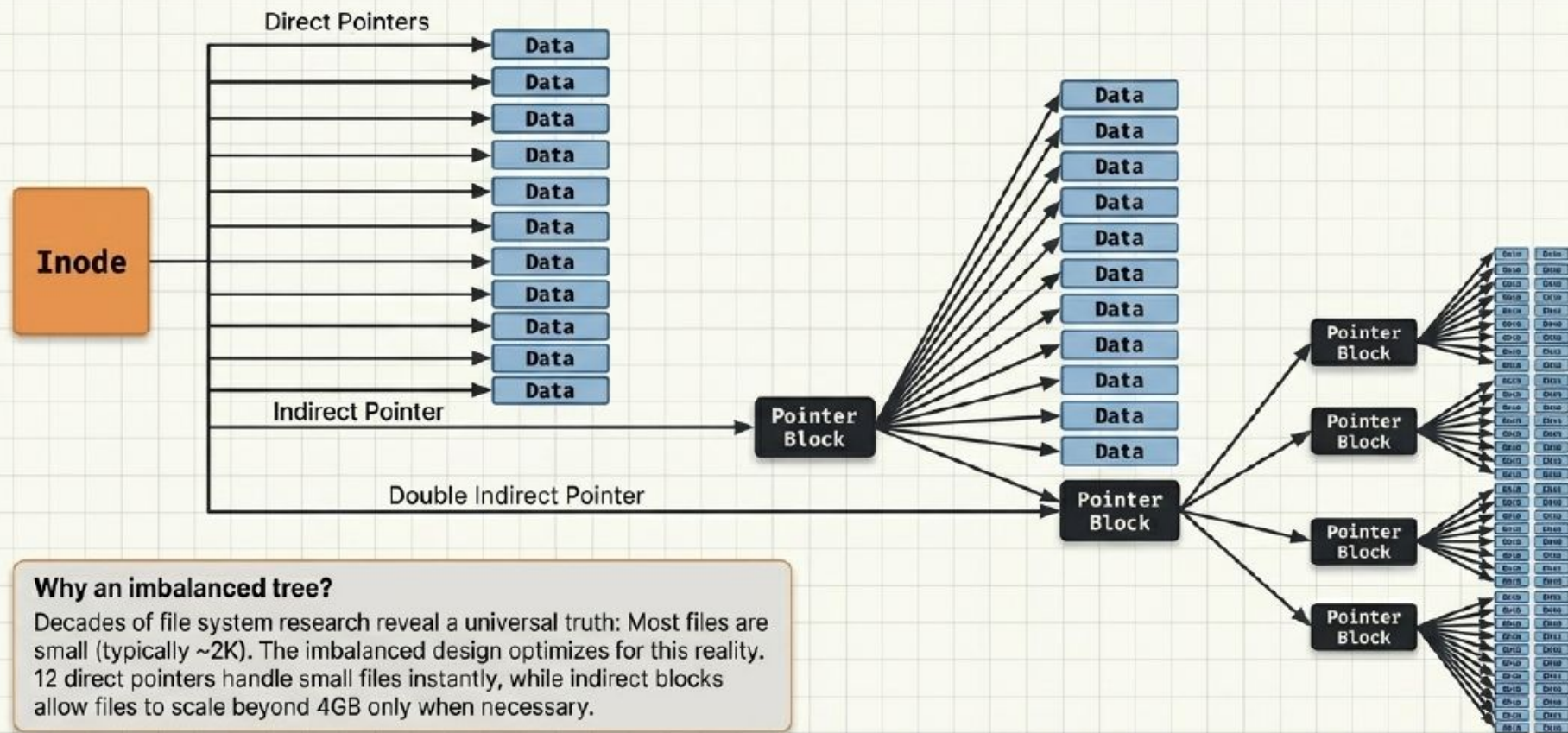


- **Super block** holds the entire organisation of all other blocks
 - Which blocks are inodes, which are data blocks, where does data block start, where Inode begins, type of file system, etc
 - During the mount, OS reads super block to initialise various parameters

Limitations on file size

- Filesystem metadata imposes limits on maximum size of file that can be stored on filesystem, maximum number of files, maximum disk size that can be managed, and so on..
- Example: limit on file size imposed by inode structure
 - Suppose inode can store K direct blocks, one single, double, triple indirect block each
 - Suppose single indirect block can store N block numbers, double indirect block can store block numbers of N single indirect blocks, triple indirect block can store block numbers of N double indirect blocks
 - Maximum file size = $K + N + N^2 + N^3$ blocks
- Different file systems differ in these limits

Scaling Data: The Multi-Level Index



Why an imbalanced tree?

Decades of file system research reveal a universal truth: Most files are small (typically ~2K). The imbalanced design optimizes for this reality. 12 direct pointers handle small files instantly, while indirect blocks allow files to scale beyond 4GB only when necessary.